

Advance - Java (Web-Java)

Java EE (Enterprise Edition)

- developed by Sun Microsystems, later managed by Oracle
- packages → Javax.*

Jakarta EE

- it's the successor of Java EE, now managed by Eclipse Foundation
- packages → jakarta.*

→ It is a set of specifications (APIs) that extends Java SE for additional enterprise features.

Ex:- Distributed Computing, Web Services, Persistence, Connection pooling, etc.

→ Designed to provide a platform for building, deploying large-scale, multi-threaded, Scalable and secure network applications.

→ What is Enterprise Application?

- An EA is a large software system platform designed to operate in a corporate environment.
- Typically it's a server-side, remotely deployed, transactional, multi threaded complex and scalable application.

Key APIs :-

- ① Web : Servlets, JSP (Java Server Page)
- ② Persistence : JPA (Java Persistence API)
- ③ Messaging : JMS (Java Messaging Service)
- ④ Transactions : JTA

Web Server vs Application Server

① Web Server is designed mainly to handle http requests/responses.

- Serves static content (HTML, CSS, JS)
- handles dynamic requests using Servlets/JSP.
- Manages HTTP req/res cycle, session management, URL mapping, page navigation, etc.

Ex:- Apache Tomcat (provides a Servlet container to execute Servlets)

① An application server provides everything a web server does, plus enterprise features to build large scale applications

Example services :- JPA, JMS, JTA, Web Services, Connection Pooling, load balancing, Security.

Examples of Application Server :-

- ① WildFly / JBoss
- ② Glassfish
- ③ WebSphere

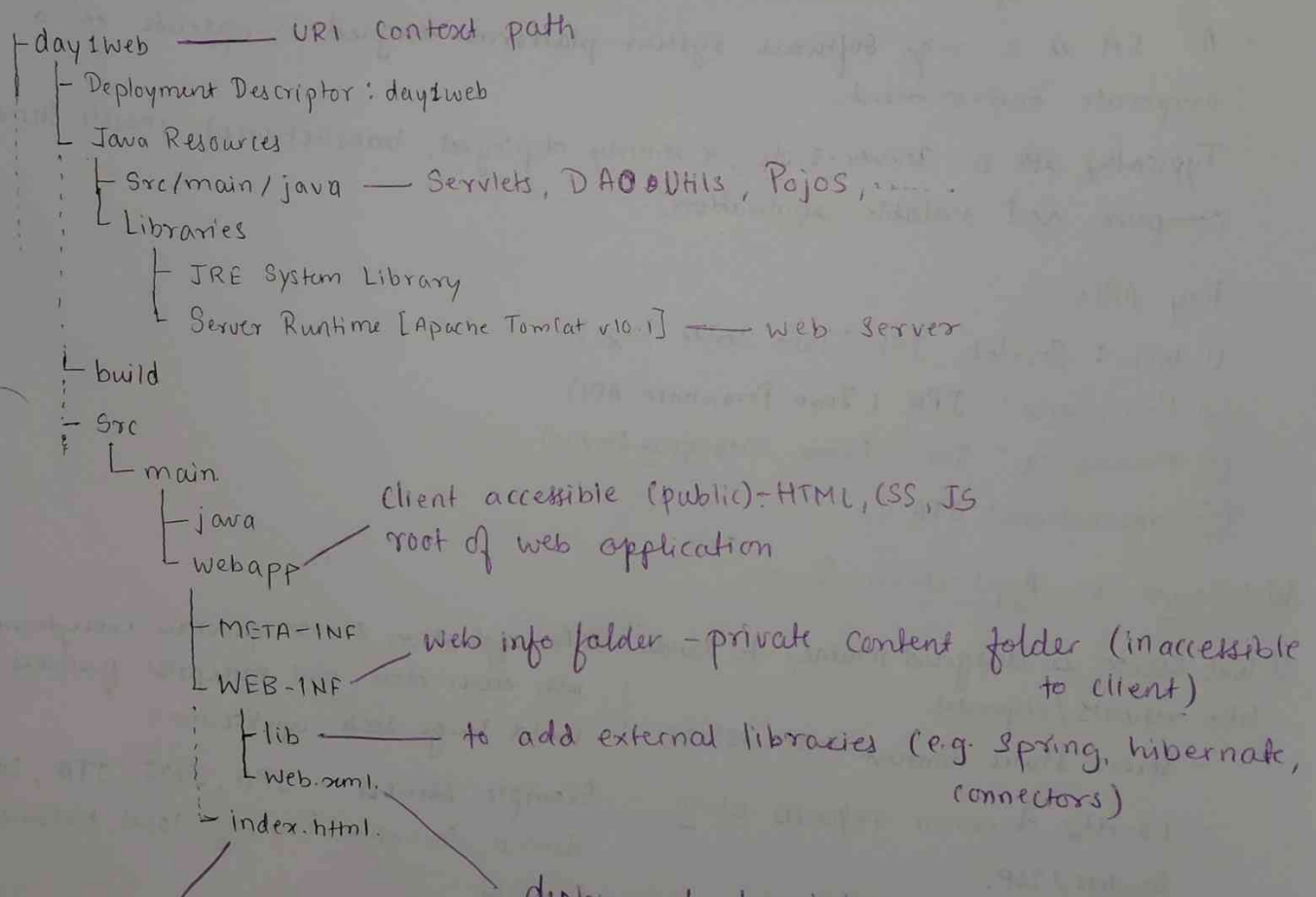
* Why Java EE ?

- offer multi-Client Support (i.e. thin client, thick client, smart client)
- Server Independence (How-Specification vs Implementation)
- Standardized Enterprise Services
- Scalable, robust, secure Infrastructure
- Interoperability with other systems.

* Dynamic Web Application ?

- a server side application, deployed on web server, meant for servicing web clients using application layer protocol (HTTP/HTTPS).

* Folder Structure



deployment descriptor

who creates it? prog. with help of IDE

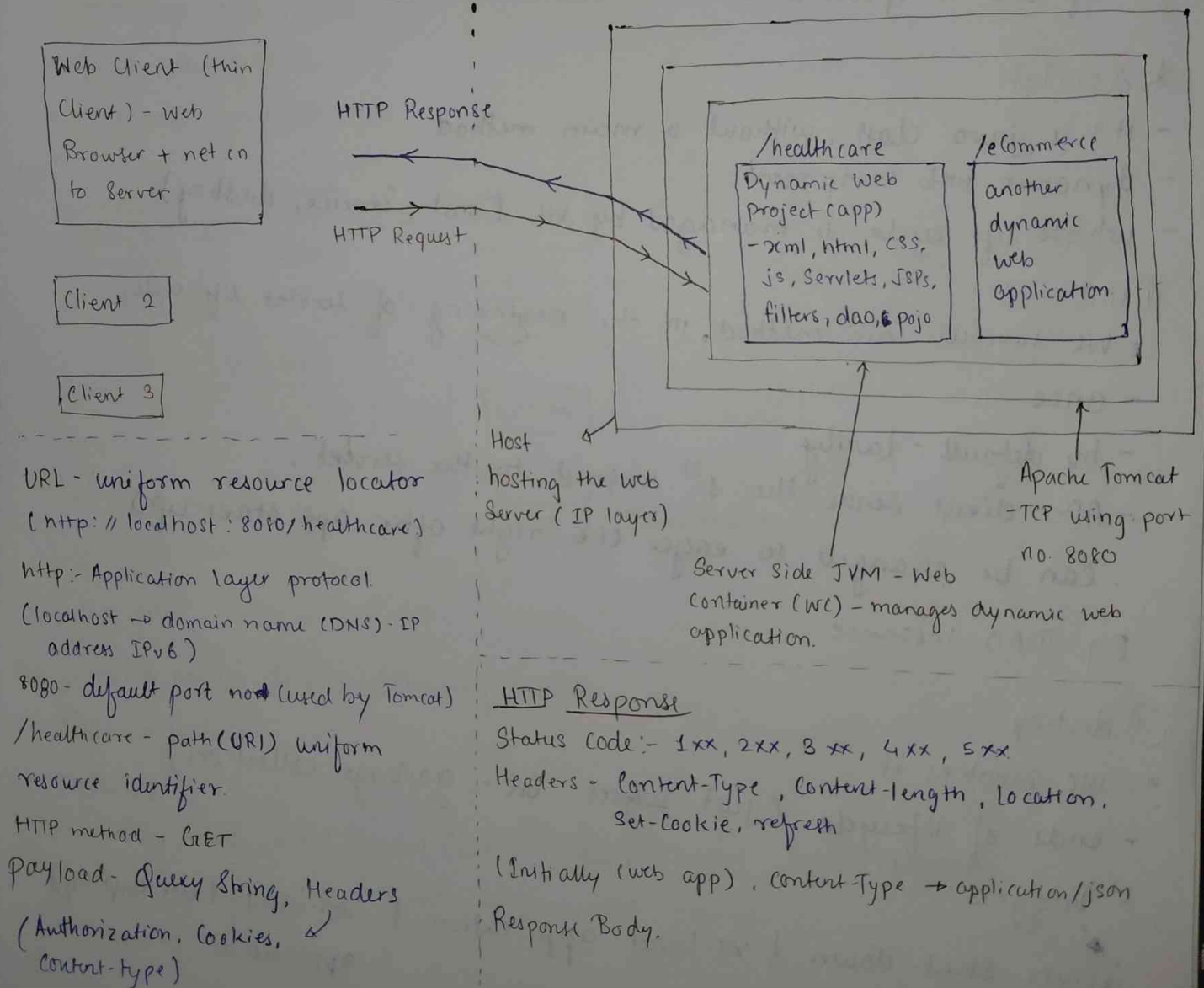
who reads it? WC (web container)

when? @ web app deployment / startup time

Contents: welcome file list, Servlet deployment tags, sec tags, ...

- Java files will go inside Java Resources > (src/main/java) folder.
- build folder is all about Compiled Java classes files.
- Web container is nothing but Server Side JVM.
- Who reads web.xml?
- WC reads this file exactly once right at Startup of application.
- Under WEB-INF, there exist a hidden folder called classes (any web app will keep the .class files inside this folder after deployment)

HTTP- Req- Res Flow



* In java web app, URI starts with Context-path
↓
Web project name.

* Web Container

- web container is the engine within a web server that provides the runtime environment for dynamic web components such as Servlets, JSPs, and filters.
- It sits inside a web server (like tomcat or jetty)
- It's responsible for managing the execution, lifecycle and interaction of web components with the client (via HTTP)

* Servlets

- It's a java class, without a main method
- dynamic web component
- whose life cycle is managed by WC (init, service, destroy)

① init

- WC invokes init method, in the beginning of servlet life cycle.
- Once
- by default - lazily
- After client sends the 1st request to the servlet.
Can be changed to eager (i.e. right after app start up)

E.g. DAO Instance

② destroy

- WC invokes it
- ends of lifecycle (just before GC - garbage collection)
- Triggers

↓
Server shut down / re-load application / un-load (un-deploy) application

E.g. DAO Clean up.

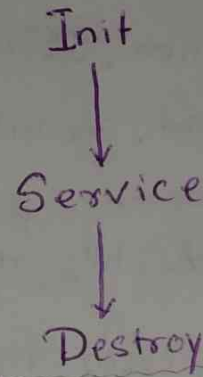
③ Service

- WC invokes it
- per every request
- E.g. DAO's CRUD methods

Why?

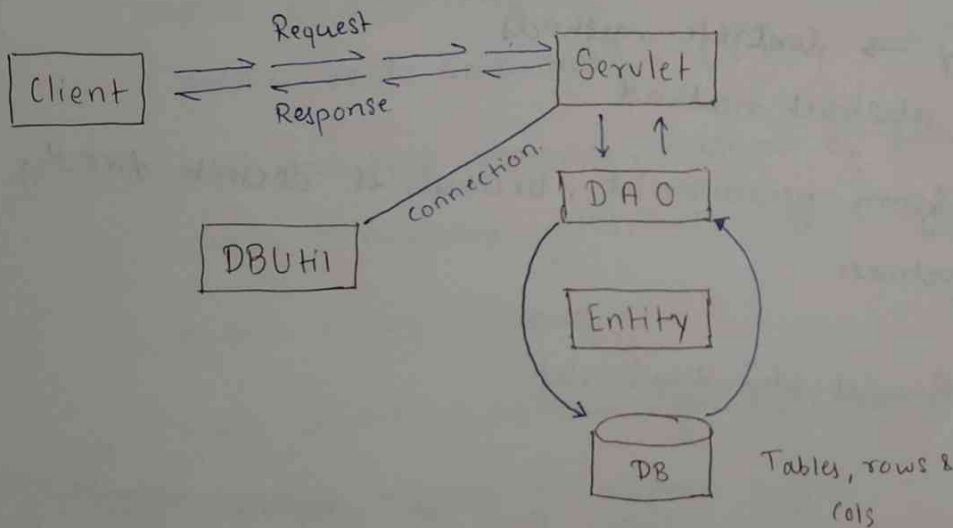
To perform jobs:-

- request processing
- Business Logic
- manage DAL / DAO
- page navigation
- response generation.



(Only one instance of servlet is created (Singleton Pattern) by WC)

*



Servlet API

1) jakarta.servlet.Servlet. (Starting point)

- Root interface

- Methods

① public void init (ServletConfig config) throws ServletException

- ServletException is raised in case init failure.

- checked Exception.

② public void service (ServletRequest rq, ServletResponse rs) throws ServletException, IOException

- jakarta.Servlet.ServletRequest
 - interface
 - represents http request
- gets called every request.
per

- jakarta.Servlet.ServletResponse
 - interface
 - represents http response.

③ public void destroy()

2) Servlet interface is implemented by

- jakarta.Servlet.GenericServlet
 - abstract class
 - init & destroy → concrete methods
 - service → abstract method
- do not extend from genericServlet, because it doesn't directly supports HTTP features.

3) Extended by jakarta.Servlet.http.HttpServlet

- abstract class
- but no abstract methods → (imp que on why no ^{concrete} ~~abstract~~ methods given in (regarding HttpServlet.txt file))
- it implements service methods & it acts like a dispatcher.
- if method = GET → call doGet();
- if method = POST → call doPost();

⋮
⋮
⋮
⋮

* Development Steps to create a Servlet

1) Create a java class which extends from HttpServlet.

2) to handle GET

- Override doGet

(protected void doGet (HttpServletRequest rq, HttpServletResponse rs)
throws ServletException, IOException)

HttpServletRequest → extends ServletRequest

HttpServletResponse → extends ServletResponse

* what if we don't override any of the doxxx methods?

→ it'll give status code 405 → i.e. method not implemented.

2.1) Set Response Content-Type.

- method of ServletResponse

(public void setContentType (String contentType);)

2.2) To send text type response, method of ServletResponse

- (public PrintWriter getWriter() throws IOException)

or to send binary response

- (public ServletOutputStream getOutputStream() throws IOException)

2.3) Add dynamic response to response buffer.

- methods of PrintWriter

print | write | println

2.4) Close PrintWriter

- response is send to the client, as soon as we close printWriter.

* How to deploy a Servlet with web container (WC)?

① Legacy Option

- using xml tags (in web.xml)

② Modern approach

- using annotations

@WebServlet ("/url-pattern") (e.g. /hello)

```
public class HelloWorldServlet extends HttpServlet {....}
```

- meaning

- We create a empty map (HashMap) @ application start up time

- We use reflection to read Servlet classes and populates the request map.

Key → url-pattern (e.g. /hello)

Value → fully qualified Servlet className (e.g. HelloWorldServlet)

URL → http:// host : port / day1-1 / hello

path → / day1-1 / hello

url-pattern → /hello

* Using xml, Servlet creation (legacy Option)

* Servlet life cycle (managed by WC)

- Start Server side web application
- WC $\rightarrow$ using reflection, creates request mapping's map.
(Key $\rightarrow$ url-pattern, value $\rightarrow$ fully qualified Servlet class name)
- (@WebServlet | XML Tags)

WC checks for load on startup.

load on startup value ≥ 0

$\rightarrow$ eager init

1) WC loads Servlet class

2) creates Singleton class instance using default constructor

3) public void init () throws SE

- init failed
(ServletException)
- Aborts life cycle

- Completes init
Sequence.

default value = -1

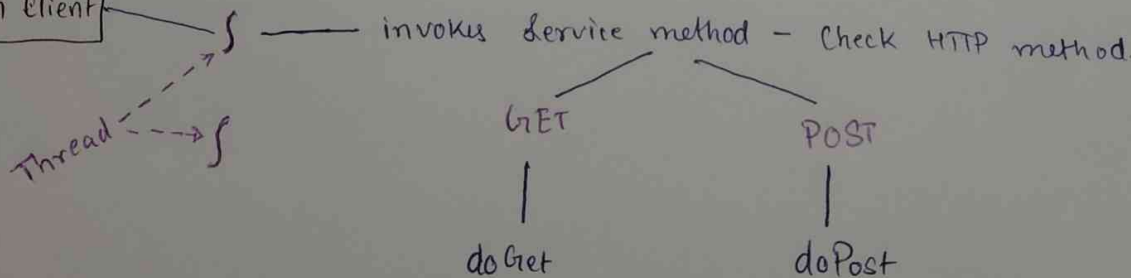
$\rightarrow$ lazy init

$\rightarrow$ waits till 1st request

& then starts the same
execution sequence of

loading, instantiating & init.

Thin Client



Programmer must override atleast one of the doxxx method,
otherwise it'll give a 405 status code

Server graceful
Shutdown

Context Reload

unload / undeploy

WC invokes public void destroy () - end of life cycle.

* Healthcare case Study

1) User Authentication.

Table - users

DButils - openConnection, closeConnection.

User Entity Class → Base DAO → here we create the cleanup logic and every other DAO will inherit from this.

User DAO - Interface

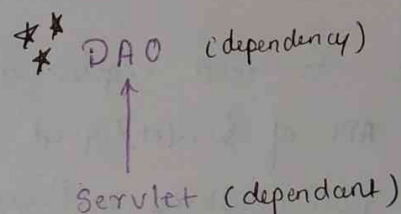
User DAO implementation - class

AuthenticationServlet - eager init

init - dao instance (create dependency)

doPost - invoke dao's method (authenticateUser(email, pass) → User

destroy - dao's cleanup



* Core-java Point

- Overriding form of the method (Sub class method) CAN NOT
- throw new or Broader Checked Exception.

* Centralised Exception Handling in Servlets

@Override

```
public void init() throws ServletException {
```

```
    // create an dao instance  
    try {  
        userDao: new UserDAOImpl();
```

```
    }
```

```
    catch (Exception e) {
```

```
        // instead of just handling/catching the exception, re-throw it back to WC  
        // so that WC aborts the life cycle, in case of init's failure.
```

```
        // API - public ServletException (String message, Throwable rootCause)
```

```
        throw new ServletException ("err in init of -" + getClass(), e)
```

```
    }
```

```
}
```

@Override

```
public void destroy () {
```

```
    try {
```

```
        userDao.cleanup();
```

```
    }
```

```
    catch (Exception e) {
```

```
        e.printStackTrace();
```

```
        // optional, but we can only throw Runtime exceptions (i.e. unchecked excp)
```

```
        throw new RuntimeException ("err in destroy" + getClass());
```

```
    }
```

```
}
```

* How to read request data from sent from client → server, ?

→ API of ServletRequest interface.

```
public String getParameter (String paramName)
```

- returns parameter value OR null if param not present.

→ protected void doPost (req, res) throws ServletException, IOException {

```
    res.setContentType ("text/html");
```

```
    // get print writer to send text resp.
```

```
    try (PrintWriter pw = res.getWriter()) {
```

```
        // read req params.
```

```
        String email = req.getParameter ("em");
```

```
        String pass = req.getParameter ("pass");
```

```
        // invoke UserDao impl. authenticate method.
```

```
        User user = userDao.authenticateUser (email, pass)
```

```
        if (!user) → means user == null
```

```
        pw.print ("<h1> Login Failed </h1>"); // failed login.
```

```
    } else {
```

```
        // Successful login to render user details
```

```
        pw.print ("<h1> Login Success </h1>");
```

```
    }
```

```
} catch (Exception e) {
```

```
    throw new ServletException ("
```


* We write prepared statements only once or multiple statements at the time of instance creation, because if we do it on demand, inside a method, it'll be slow as we are preparing statements for each incoming request.

* Page Navigation Technique

- Taking the client from one page → another page

1) Client Pull.

- client is involved in navigation

1.1) User taking action (e.g. clicking on link, button, etc)

- Web browser will send a new URL request and this will take to the new page.

★★ 1.2) Redirect Scenario (imp)

API of HttpServletResponse

- public void sendRedirect (String redirectLocation) throws IOException

E.g. response.sendRedirect ("page-name");

→ We don't write "/" before url.
°° it omits the context path (imp)

Internals

1) WC sends temporary redirect response.

- Status Code (302 - redirect), Headers - Location: "patient-dashboard",
response body: EMPTY

2) Web Browser sends next REDIRECT request with URL

(http://host:port/context-path/redirect-location)

(day2-1) (patient-dashboard)

Method - GET

★★★ Whenever the redirect is done, the WC does is, it clears the response buffer.

(IMP Que)

* When will web container throw IllegalStateException ?

- If you have already sent the response (e.g. pw.flush or pw.close) & then called sendRedirect.

Why? → because WC tries to clear response buffer at SendRedirect,

- WC can't clear the buffer, when the response is already sent !!!

* Problem

- Web application was unable to remember client details in dashboard page. (null values)
- (Servlet)
- ~~When~~ In redirect scenario, client is navigated to next page in the new/next request.
 - HTTP is stateless $\rightarrow$ i.e. it cannot remember client information / state across multiple requests, and.
 - $\rightarrow$ it cannot ~~be~~ identify the client state separately

* Server side State Management Technique (Java EE)

① Cookie

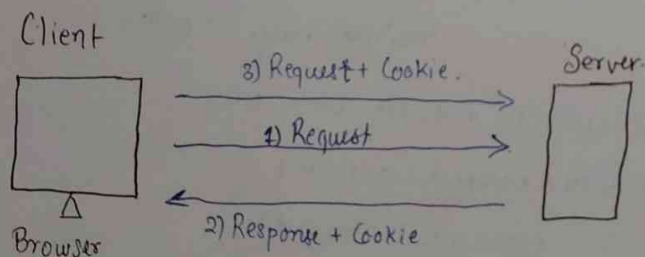
or

② ~~HttpSession~~ HttpSession (interface)

or

③ HttpSession + URL rewriting (encoding)

* Cookie Based State Management



1) Create a cookie

API $\rightarrow$ `javax.servlet.http.Cookie (String nm, String val)`
(jakarta)

`public Cookie (String name, String value);`

2) Add cookie in response header

API of `HttpServletResponse`

`+ public void addCookie (Cookie c);`

→ web browser checks if the cookies are blocked,

→ if yes, can't remember the user/client.

if no,

cookies are stored in browser's cache.

→ web browser auto adds the cookie/s to any further request made to same website (dynamic web application).

3) To get the cookies from request headers

API of `HttpServletRequest`

→ `public Cookie[] getCookies();`

- returns Null, in case of no cookies

* To extract the value of a cookie / name of a cookie.

API →

① `public String getName` → to get name of cookie

② `public String getValue` → to get value of cookie

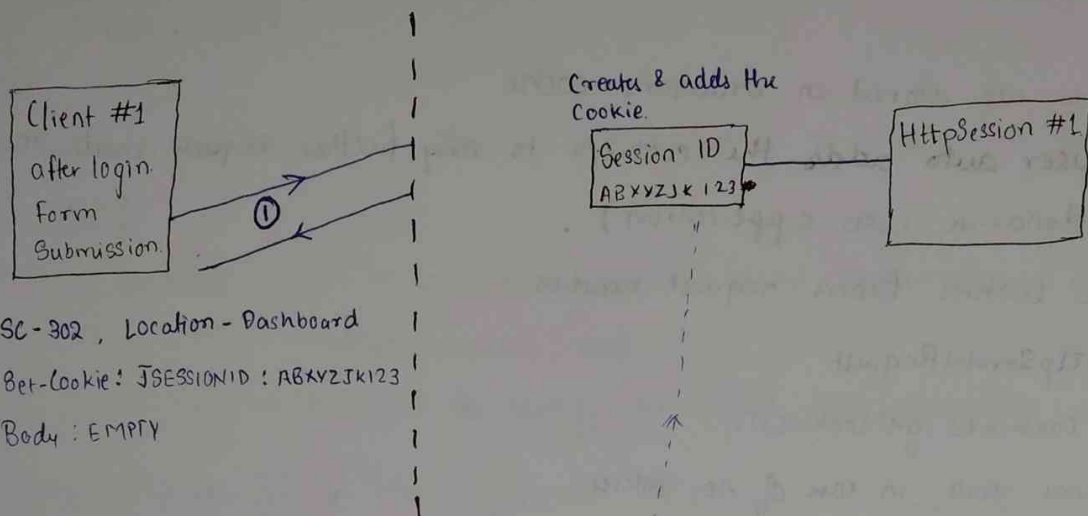
* Cookies → A small text data/information sent from Server to client on login to track sessions or state management.
(Servlet) (browser)

→ the browser saves the cookie and later sends these cookies in each request back to the server.

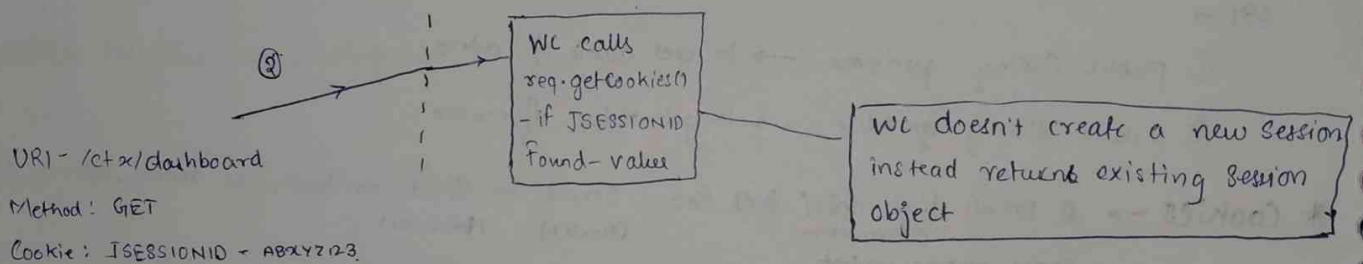
Disadvantages of Cookies:-

- ① Developer has to manage cookies
- ② Cookies can handle only text data. Storing java obj. or binary data is difficult.
- ③ As no. of cookies ↑, it'll result into increased network traffic
- ④ WC (cookie processor) may not directly support all the characters, so programmer has to encode & decode
- ⑤ In cookie based approach → if client rejects cookies, state will be lost ∴ Session tracking fails.

* HttpSession Based State management



* Cookie is stored in browser's cache



* HttpSession -

Stores the client side state using the (setAttribute & getAttribute) methods.

→ HttpSession API

- State stored on the server side & inside a HttpSession Object.
- Only a Session ID is stored on the client (typically in a cookie like JSESSIONID)
- Servlet Container (WC) manages the cookies automatically.

Development Steps

① Get the HttpSession Object from web container

- Method of HttpServlet Request

public HttpSession getSession()

- In case of new user, WC creates NEW HttpSession obj. & returns it to caller.
- In case of existing user, WC returns EXISTING HttpSession obj. to the caller.

Ex:- HttpSession getSession = request.getSession();

② To save client details under HttpSession

→ HttpSession API

- public void setAttribute (String attrName, Object attrValue)

e.g. session.setAttribute ("patient-details", patient);

③ To get client details from HttpSession

→ HttpSession API

- public ~~void~~ Object getAttribute (String attrName)

e.g. Patient patient-details: (Patient) session.getAttribute ("patient-details");

④ To discard HttpSession (typically in 'logout' page)

→ HttpSession API

public void invalidate ()

Extra APIs

⑤ To remove attribute from HttpSession

→ public void removeAttribute (String attrName) ;

⑥ To get all attribute names bound to HttpSession.

→ public Enumeration<String> getAttributeNames () ;

For Debugging

⑦ public boolean isNew ()

- returns true in case of newly created HttpSession.
- otherwise false.

⑧ public String getId ()

- Returns HttpSession Id, created by WC, unique per client.

⑨ Default value of HttpSession expiration time, for apache tomcat = 30 mins
To change its value.

public void setMaxInactiveInterval (int seconds).

OR, using (declarative method) xml-tags

```
<session-config>
  <session-timeout> timeout in minutes </session-timeout>
</session-config>
```

* In hibernate, when doing ORM b/w user & patient, we'll have a has-a relationship b/w user & patient & for patient's foreign key user_id, it'll have the User Object, in patient's Object. just like in prisma, we have a todos[] array in users & a user object in todos object.

```
User {
  ...
  Todos[]
}

Todo {
  ...
  User
}
```

* DTO → Data transfer Object

Object which is used to transfer the data b/w the layers, for example. when we have appointment details and doctor's name, so we ~~need to~~ need to create a custom class like AppointmentDTO, which will have both the required fields

- Entity vs DTO

Entity represents our database table.

- DTO represents customized view, as well as data aggregated / fetched from multiple tables.
- To transfer the data b/w the layers
- In current example DAO returns list of appointmentDTOs → to caller (Server)
Later → it'll be used to exchange the data b/w REST client & REST server.
OR
Between the microservices

* Select appointments of a patient, doctor name

→ Select a.appointment_id, a.appointment_date_time, u.firstname, u.last_name
From appointments a join doctors d on a.doctor_id = d.doctor_id
join users u on d.user_id = u.user_id
where a.patient_id = ? and a.status = "SCHEDULED" and
a.appointment_date > now() order by a.appointment_date_time asc;

* Development Steps for Patient Dashboard

① Get patient details from userId. (from HttpSession)

② Get list of upcoming appointments.

1.1) PatientDao extends BaseDao

- Get patients details from user Id.

Patient getPatientDetailsFromUserId (long userId) throws SQLException

2.1) Appointment Dao - extends BaseDao

- Get list of upcoming appointments

List<AppointmentDto> getPatientsUpcoming Appointment (long patientId) throws
SQLException;

③ In patientDashBoard servlet

-- init

- patient Dao, appointment Dao

- destroy

- cleanup

- doGet

- get user details from session

- call patient Dao's method to get patient details

- call appointment dao's method get list.

- render patient details from list.

④ Book Appointment

4.1) Appointment DAO

- String bookAppointment (long docId, long patientId, Timestamp ts)
throws SQLException;

4.2) Implementation

- check for doctor's availability

Sql:- Select appointment_id from appointments where doctor_id = ?
and appointment_data_time = ? ;

- if available

Create appointment entity class instance

- insert query

- Success message.

⑤ Patient Dashboard Servlet

- add a link to book appointment

→ upon click, take patient to HTML Form for appointment booking.

- accept doctorId, Timestamp → Submit the form to

- AppointmentServlet : Singleton & eager

@Override

- init → create appointmentDao instance

- destroy → cleanup of dao's

- doPost → ① read request parameters

- docId : paramLong

- timestamp : ???

② get patient details from Session → patient Id.

③ create appointment entity instance n then call dao's method.

④ Redirect client to dashboard & render message.

* Timestamp: YYYY-MM-DD HH:MM:SS

LocalDateTime: YYYY-MM-DDT HH:MM:SS

↓ String → LocalDateTime

LocalDateTime dateTime = LocalDateTime.parse(req.getParameter("appointmentDateTime"))

Timestamp ts = Timestamp.valueOf(dateTime);

↓

~~LocalDateTime~~ LocalDateTime → Timestamp

* Troubleshooting Techniques

① Undeploy all unused applications

> right click on servers > clean...

② rebuild project

> project > clean... > [Select project to

Servlet Request API

* public String[] getParameterValues()

- returns String[] of request parameters value.

- use case → check box, multiple selection list.

* Cancel Appointment

① Add cancel link in PatientDashboard.

② When (Client) clicks on cancel button →

http://localhost:8080/day4-1/appointments?id=8

Steps

* Servlet Config

- jakarta.servlet.ServletConfig (Interface)
- to represent servlet specific configuration
- Created by WC, one per Servlet, after creating the Servlet instance (via default constructor) & before calling the init method.

use case :-

- for using servlet specific initialization parameters

e.g. to create DB independent web application

* WC always calls default constructor, but if we write a parameterized constructor inside Servlet, then the compiler won't give the default constructor and the WC can't instantiate singleton Object & it fails.

* Development Steps

- Add DB specific parameters (url, username, password) in web.xml file or in annotations.

① web.xml file (always a better way to add init parameters)

```
<init-param>
  <param-value> ... </...>
  ...
</init-param>
```

② WC starts the servlet life cycle.

- Servlet class loading
- Creates singleton instance using default constructor.
- Creates ServletConfig object & populates it with init-parameters, if any. (Dependency injection) ↗
- Passes ServletConfig object to ~~Servlet~~ Servlet in the init() method.

* How to get, servlet specific init params from init method ?

① API

Get Servlet Config → `public ServletConfig getServletConfig();`

② API for ServletConfig Object

- public String getInitParameter (String paramName)
- Returns the value
- returns null - if param is absent.

* To add init params in annotations.

```
@WebServlet (value = '/login', loadOnStartup = 1, initParams = { @WebInitParam (
                                                                    name = 'nm', value = 'val') })
```

X

* What is ServletContext ?

- interface jakarta.servlet.ServletContext
- Specifications are in jakarta provided JAR - servlet-api.jar
- Apache Tomcat Implementation - catalina.jar
- ServletContext represent an interface in WC
 - Represents the web application
 - WC creates the ~~application~~ instance of ServletContext, at the web app start (deployment) only once per web application

→ Use case

→ To create global parameters (i.e. scope = current web app)

in web.xml (Similar to init params)

```
<context-param>
```

```
  <param-name>...</>
```

```
  <param-value>...</>
```

```
</context-param>
```

API

① Get Servlet Context

```
public ServletContext getServletContext ()
```

Method of

② ~~Get~~ ServletContext interface

```
public String getInitParameter (String InitParamName)
```

- returns param value if present, otherwise null.

* Parameters are Read-only and immutable, which makes them Thread-Safe, meaning no locking required.

Parameters type is (String, String)

* Attributes are Read & Write and mutable, which makes them Not-Thread Safe, meaning locking is required.

^{Attributes}
~~Parameters~~ type is (String, Object)

→ Context Params

- read only
- inherently thread safe
- locking is not required

(Synchronization → applying ~~read~~ thread safety = locking)
when it is required?

In a multithreaded application

- multiple threads are sharing the common resource
- Some threads are reading, others are updating (writing)

→ Context Scoped Attributes

- 1 per entire web application
- readable & writable (mutable)
- inherently thread un-safe
- Access it using synchronized block

e.g. In ecommerce application (How to add same info?)

- add it as ctx scoped attribute.

```
Synchronized (getServletContext())  
{
```

```
// access | modify the attribute here....
```

```
}
```


* Server side log

→ method of ServletContext i/f
public void log (String message)

* Request Dispatching | Request Chaining | Server Pull

(Page navigation technique ②)

- used commonly in MVC based architecture
- Navigating the client to next page, BUT in the same request.
- Dispatching the same req to visit multiple pages (HTML/JSP/Servlet)
- Faster than Client Pull (as round trip of request get's eliminated)

Why?

- Separation of Concerns (tasks/responsibilities)
- Request Dispatching | Chaining.

API

1) Get RequestDispatcher (interface - jakarta.servlet) from either ServletRequest or ServletContext

→ ServletRequest Method:-

public RequestDispatcher getRequestDispatcher (String path);

- Methods of -RequestDispatcher

mostly used ① public void forward (ServletRequest request, ServletResponse response);

② public void include (ServletRequest req, ServletResponse res);

* Minimum Scope required in Server Pull is Request Scope.

* Minimum Scope required in Client pull is Session Scope.

* in Server pull as we are doing all the processing / page navigation in single Request, then we only need request scope as the same request will be passed / used throughout.

→ So, to set attribute in request.

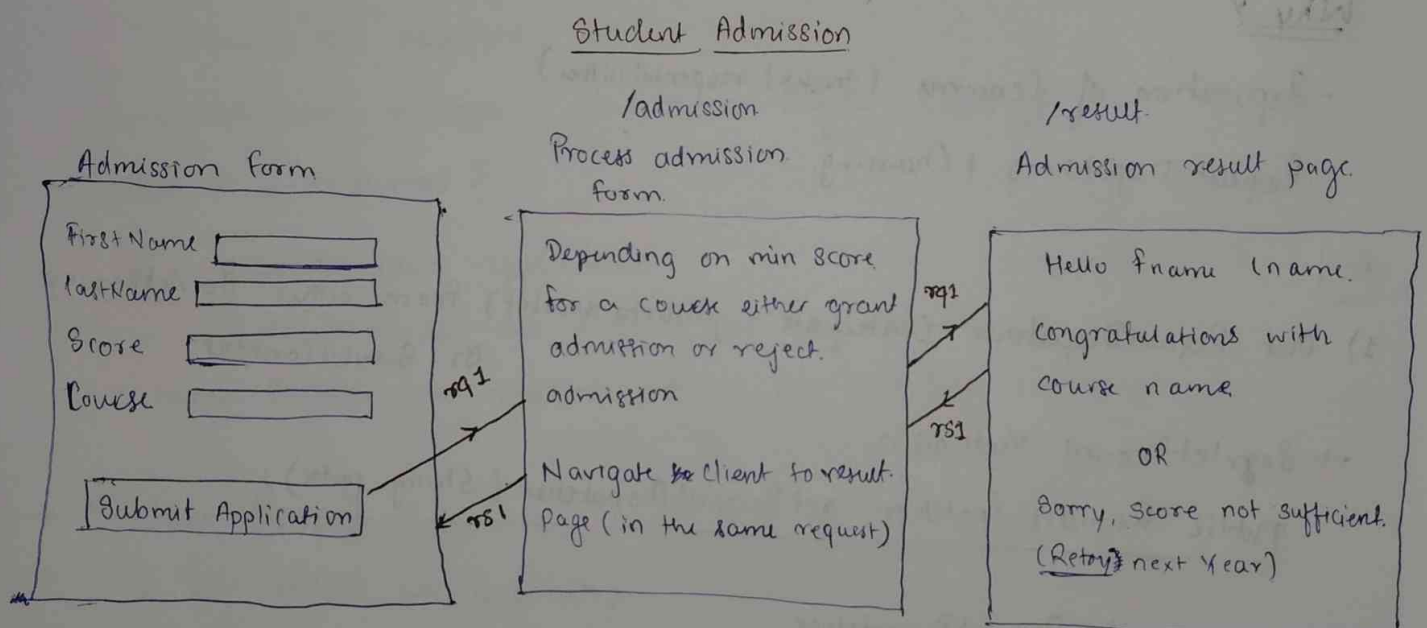
- request.setAttribute("Student details", Student);

→ get the dispatcher

- RequestDispatcher rd = request.getRequestDispatcher("result");

→ forward the Client to the next page in same request.

- rd.forward(request, response);



* doGet → forwards / includes → doGet

* doPost → forwards / includes → doPost

* Internals for rd.forward (rq, rs)

- ① WC clears (discards) the response buffer
- ② WC invokes doxxx method of the next page (doGet → doGet → doGet → ...)
or
(doPost → doPost → ...)
- ③ Only the last page in the chain, can generate the dynamic response
(i.e. it can add to the response buffer)
- ④ After the method exec, control comes to 1st page.
- ⑤ response is then rendered (sent to client)
- ⑥ This is the default technique used for navigation in typical MVC framework.

* When will WC throw IllegalStateException?

→ If we call forward, after committing the response.

Forward vs Include

- both are request dispatching techniques
- min scope reqd. = request

Difference

In Include Scenario :-

- main page, as well as include page, both can generate dynamic contents
(i.e. pw.print)
- The include page cannot set response status code & Headers

* Internals for rd.include (rq, rs)

- ① WC does not clear/discards response buffer
- ② Same as Forward.
- ③ main page as well as intermediate / last page can also add to the dynamic response.

(All other points same)

(Tmo)

* Web Application Event Handling

- in a java/ web application, multiple events are taking place.

e.g. ServletContext are getting created & destroyed.

Attributes are getting added, modified, removed.

Servlet Request & response are getting created & destroyed objects.

→ To handle these events, java EE supplies Event Listener Interfaces.

e.g. ServletRequestListener, HttpSessionListener, ServletContextListener, HttpSessionAttributeListener.....

methods of ServletRequestListener

① public default void requestInitialized (ServletRequestEvent e)

② public default void requestDestroyed (ServletRequestEvent e)

* Development Steps

1) Create a class that implements

- listener interfaces

2) To tell WC, following is an event listener class

@WebListener - class level annotation

OR

in web.xml

<listener>

<listener-class> F.Q listener class name </...>

</listener>

WC right at Server Startup Creates a Thread Pool with some size, when a request comes, the WC takes a thread & services/Processes the request and returns the thread to the pool.

* How does WC handle the concurrent request handling from multiple clients?

→ It doesn't use thread per request pattern, since for multiple requests, we will need to create a separate thread & destroy it, when the response is rendered.

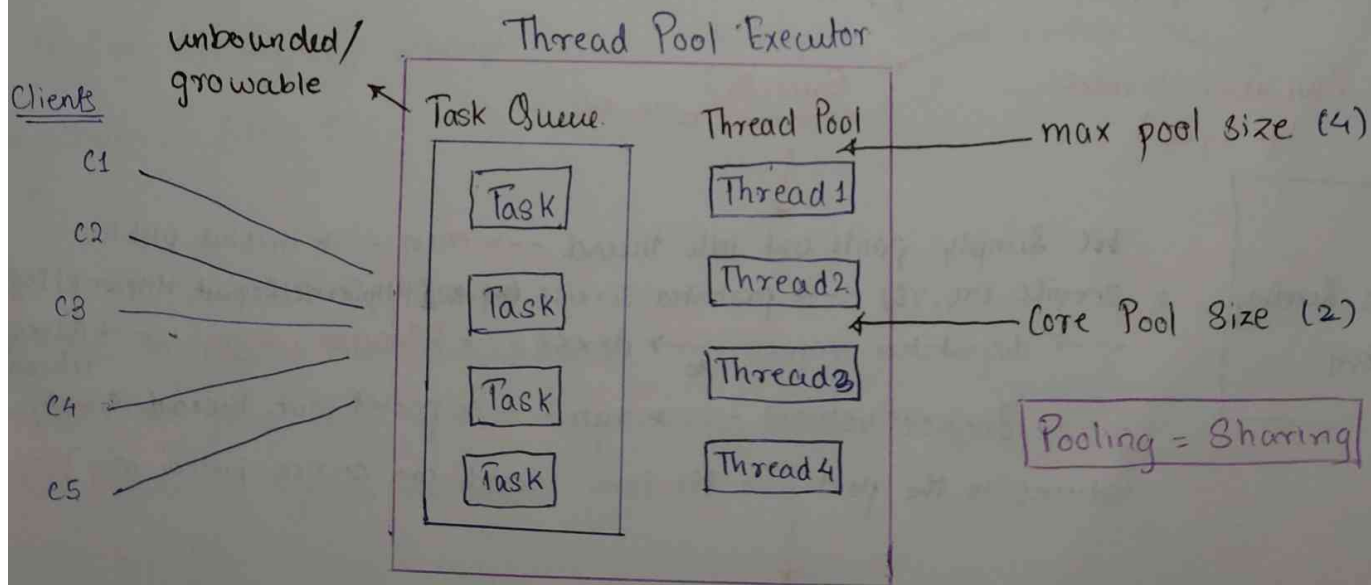
→ Instead it uses thread pool.

- Internally it uses Executor framework, supplied from `java.util.concurrent` package.

Why?

- Re-usability of servicing threads
- Resulting in better scalability.

* Thread Pool Based Executor used internally by Web Containers



Lifecycle of Servlet (managed by WC)

@WebServer Start up: WC Starts up - creates thread pool
(core size n max of the thread pool, will be as per the default OR can be configured from xml files.)

deployment

@Web app dep time

① WC creates the instance of ServletContext per web app

② Prepares Servlet url-mapping (key - url-pattern, value - F.Q. Servlet class name)

WC checks for load-on-startup

load-on-startup > 0 → eager

(Starts Servlet @ web app dep time)

① Loads Servlet class (Server side method area)

② Creates Servlet instance: using default ctor

③ WC creates ServletConfig instance, populates it with init params.

④ Invokes public void init() throws ServletException;
(WC implicitly passes ServletConfig obj. to init)

Failure (WC aborts Servlet life cycle)

Success?
Continues with 1.6.

load-on-startup = -1 → lazy

WC waits for 1st request from any client - then start the life cycle (same init seq.)

Clients Sending Requests

WC simply pools out idle thread. → run → invokes public service (req, rs) → protected service (HttpServletRequest, HttpServletResponse) → dispatches request to doxxx → { Servicing logic } → doxxx returns → service returns → run → pooled out thread simply returns to the pool (So the same thread can service further req.)

Triggers

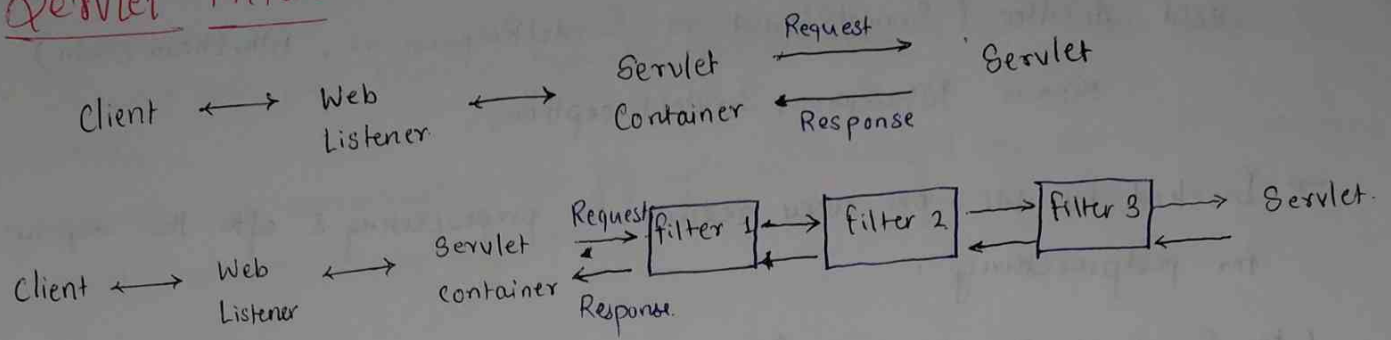
Server Shutdown

Web app reloaded

Web app undeploy

WC invokes public void destroy() (mainly used for cleaning up of resources) then WC marks the Servlet instance for GC - ending the Servlet life cycle.

Servlet Filters



(filters are like middlewares in node.js)

* Filters are used to do preprocessing and postprocessing

* Why Servlet Filters?

→ ① Provides re-usability

meaning- provide the ability to encapsulate recurring tasks (cross cutting concerns) in reusable units

They provide clear cut separation b/w Business Logic & cross-cutting concerns

e.g. authentication, authorization, etc.

② Can dynamically intercept req/res to dyn or static content.

* What is Filter?

→

* How to create Filter Component?

① Create java class which implements Filter Interface.

② Implement lifecycle methods

- init

public void init (FilterConfig filterConfig) throws ServletException.

(It is called by WL, only once during filter creation & initialization)

(@applicatⁿ startup time.

2.2) doFilter

void doFilter (ServletRequest rq, ServletResponse rs, FilterChain chain)
throws IOException, ServletException.

→ Invoked by WC - per every request for preprocessing & after the response for postprocessing.

(to forward ~~the~~ to next filter, we invoke chain.doFilter which passes control to next filter)

2.3) destroy()

public void destroy()

→ invoked by WC at the end of filter life cycle.

Triggers :- Server shut down / re-deploy / un-deploy

* How to deploy filter component ?

① Annotation

→ @WebFilter (url-patterns = {"/secure"})

② XML Tags

→ <filter>

<filter-name> abc </...>

<filter-class> filters.AuthFilter </...>

</...>

<filter-mapping>

<filter-name> abc </...>

<filter-pattern> /* </...>

</...>

* JSP (Java Server Page)

- Dynamic web page (having typically HTML5 markup) can contain java code directly.
- Dynamic web component, whose life cycle is managed by WC (JSP container/Servlet Container/Servlet Engine)

* Why JSP ?

- ① JSP allow developers to separate presentation logic (dynamic response generation) from business logic or data manipulation logic.
- ② Typically JSPs - used for Presentation Logic.
- ③ Ease of development - JSP pages are auto, translated by WC in to Servlet & compiled & deployed.

Note:- JSPs will not be used in the Full Stack App.

* JSP API - is a part of Java EE (Jakarta EE) Specifications

jsp-api.jar ----- <tomcat>/lib : Specs

Contains Tomcat's JSP API Implementation classes : Jasper.jar.

JSP API

Specs :- jsp-api.jar

Tomcat :- jasper.jar
impl

jakarta.servlet.Servlet

jakarta.servlet.jsp.JspPage - interface of Servlet

Methods :-

public void jspInit()

public void jspDestroy()

↓
jakarta.servlet.jsp.HttpJspPage - Sub Interface
public void _jspService (HttpServletRequest rq,
HttpServletResponse rs) throws ServletException,
IOException

↓
Implemented by Web Server (WC)

Tomcat - jasper.

JSP Life Cycle

index.jsp → 1) Translation Phase

.jsp → .java



2) Compilation Phase

.java → .class (translated page class)



3) init Sequence over

Class Loading → Singleton Instance → ServletConfig Object

public void jspInit()



4) Servicing / Run-time / Request Processing Phase

per every request → we simply pool out a thread from thread pool

→ we run() / call() → `_jspService(req, res)` (i.e. code from scripts, expressions, HTML templates) → after its exec → `_jspService` returns

→ run returns → pooled out thread simply returns to pool (so that same thread can be REUSED to service other request) (for ↑ Scalability)



5) At the end of JSP life cycle - we invoke public void jspDestroy()
→ before marking the JSP page class instance for GC

(Triggers → Server shut down / reload web app / undeploy web app)

* JSP (2 x 13 x) Syntax

① JSP Comments

1.1) Server-Side Comment

Syntax:- `<%-- comment txt --%>`

(translator & compiler ignores the text)

1.2) Client-Side Comment

Syntax:- `<!-- comment txt -->`

(translator & compiler do not ignore text)

② JSP Implicit Objects (available only to `_jspService()` available from Scriptlets & Expressions.

Object

Datatype

2.1) out

`jakarta.servlet.jsp.JspWriter` (Similar to `PrintWriter`)

2.2) request

`HttpServletRequest`

2.3) response

`HttpServletResponse`

2.4) config

`ServletConfig` (used for init parameters)

2.5) session

`HttpSession` (by default all JSPs participate in session tracking i.e. obj. is created)

2.6) application

`ServletContext` (used for request dispatching)

2.7) exception.

`java.lang.Throwable` (available only to error handling pages)

(rarely used) 2.8) page

Current translated page class instance created for 'this'

JSP

`PageContext` (this class stores references to page specific objects)

Usage:- 1) To access other implicit objects (exception, out, config, session)

e.g. `pageContext.getSession()` $\rightarrow$ `HttpSession`

2) To add/get private (page scoped) attributes, available only to current page.
e.g. `pageContext.setAttribute("string nm", object val)`

`pageContext.getAttribute("string name")` $\rightarrow$ attribute value

③ Scripting Element

3.1) Scriptlets

* Syntax:- `<% Java code written inside here %>` : within body tag
(there can be multiple scriptlets & it gets translated once per request)

3.2) JSP Expression

* Syntax:- `<%= expression to evaluate %>`

→ Evaluates to an expression → result is converted to string -- send to client browser.

eg. `<%= new Date() %>`

(Expression should return a value or expression should evaluate to a value)

(But if it returns a void, Compilation Error is thrown)

(Imp) More concise alternative to JSP Expressions (for accessing attributes) : EL Syntax

Syntax:- `${EL Syntax}`

* Important EL Implicit Objects

Name	Significance
1) Param	- map of request parameters eg. <code>\${param.get("email")}</code> or <code>\${param.email}</code>
2) pageScope	- map of page scoped attributes. eg. <code>\${pageScope.attrName}</code> or
3) requestScope	- map of request scoped attributes
4) sessionScope	- map of session scoped attributes
5) applicationScope	- map of application scoped attributes
6) pageContext	- instance of PageContext's sub class in jasper.jar
7) cookie	- map of cookies (cookie Objects)
8) initParam	- map of ServletContext Params

Example of EL Syntax

→ `#{abc}` -- We start searching in page → request → session → application → null
↓
Points blank client browser ← -- attribute is absent

→ Suppose you have to add context scoped attr ~~is~~ - loan-scheme

① in Servlet

```
getServletContext().setAttribute("loan-scheme", Scheme);
```

② in JSP

```
<%  
    application.setAttribute("loan-scheme", Scheme);  
%>
```

3.3 JSP Declarations (Private members of the translated page class)

Syntax:- `<%! JSP Declaration Block %>` (outside body tag)

Usage:-

- 1) Creating page scoped java variables & methods (instance vars & methods & static members)
- 2) Also can ~~also~~ be used for overriding life cycle methods (jspInit, jspDestroy)

* If we change method from GET → POST or vice-versa, no changes are needed to be done in further JSP as, only -jspService gets called (i.e. no doGet, doPost are present here)

* JSP Directives

- Command meant for JSP Engine (WC) to be processed during translation Phase

Syntax

<%@ DirectiveName attribute list %>

① Page Directive

- all commands meant for current page only.

Imp page directive attributes

~~import~~

Attrib. Name

import

language

contentType

session

errorPage

isErrorPage

Value ~~java.lang.LocalDateTime~~

java.lang.LocalDateTime

java

text/html

true/false (default = true → indicating we supplies HttpSession Object to all JSPs)

path of a error-handling page

true/false (default = false, → if enabled to true, we will supply 'exception' & 'err data' from this page)

<%@ page errorPage = 'err-handler.jsp' %>

one.jsp

Centralized Exception

<%@ page isErrorPage = true %>

err-handler.jsp

We supply additionally
- exception
- err data

EL Syntax for getting error message
error status code

{ pageContext.exception.message }

{ pageContext.errData.statusCode }

- Instead of adding error page attribute in multiple JSPs, add

```
<error-page> xml tag in web.xml  
  <exception-type> java.lang.Exception </...>  
  <location> /my_err_handler.jsp </...>  
</error-page>
```

* Page

- JSP Implicit Object
 - can be accessed using scriptlet & expressions
 - Instance of current translated ~~java~~ page class instance
- eg. test.jsp > test.jsp.java > class: test.jsp - instance of test.jsp class.

* PageContext

- accessible from scriptlets, expressions & EL syntax.
- instance of jakarta.servlet.jsp.PageContext.

- usecase

- ① used to access all other implicit objects.
- ② to add page scoped attributes

eg. <%
 pageContext.setAttribute("abc", 1234);
 Integer val = (Integer) pageContext.getAttribute("abc");
 out.print(val);
%>

* PageScope

- EL Implicit Object
- map of page scoped attribute

eg. \${pageScope.abc} or \${abc}

- Instead of adding error page attribute in multiple JSPs, add

```
<error-page> xml tag in web.xml
  <exception-type> java.lang.Exception </...>
  <location> /my_err_handler.jsp </...>
</error-page>
```

* Page

- JSP Implicit Object
 - can be accessed using scriptlet & expressions
 - Instance of current translated ~~java~~ page class instance.
- e.g. test.jsp > test.jsp.java > class: test.jsp - instance of test.jsp class.

* PageContext

- accessible from scriptlets, expressions & EL syntax.
- instance of jakarta.servlet.jsp.PageContext.
- usecase

① used to access all other implicit objects.

② to add page scoped attributes

```
e.g. <%
    pageContext.setAttribute("abc", 1234);
    Integer val = (Integer) pageContext.getAttribute("abc");
    out.print(val);
%>
```

* PageScope

- EL Implicit Object
- map of page scoped attribute

e.g. `${pageScope.abc}` or `${abc}`

* URL Rewriting + Http Session

Problem:- HttpSession ~~and~~ based session tracking fails in the absence of cookies.
(Since WC cannot get session id from client in form of cookie -

JSESSIONID)

Solution:- Instead of using cookie based session id, embed it in every outgoing url.

* What is URL rewriting?

- Embedding session id in the URL

* Servlet API for url rewriting

Methods of HttpServletResponse

1) public String encodeURL (String url) (Client Pull) (Scenario)

- WC encodes the URL with value of session ID, if required
- Otherwise suppresses URL encoding

Use case:- in case of clicking on the link/btn (i.e. user action is involved)

2) public String encodeRedirectURL (String url)

- WC encodes the URL with value of session ID, if required.
- otherwise suppresses URL encoding.

Use case:- in case of redirect scenario.

* Any Problems?

- vulnerable to security attacks
- Session Stealing (session fixation) attack.

* Solution:-

- Always use https instead of http, along with the url rewriting (to ensure end to end encryption)

* include Directive

`<%@ include file = "URI of the page to be included" %>`

- Typically used for including static content; —
but can include dynamic contents as well.

e.g. `<%@ include file = "2.jsp" %>`

* JSP Actions

- Commands for WC to be applied at runtime.

→ Standard Actions

Syntax: `<jsp:actionName attribute list>`

nested tags

`</jsp:actionName>`

OR `<jsp:actionName />`

* Actions

1) `<jsp:forward page = "uri of next page" />`

- used for request dispatching
- forward scenario

2) `<jsp:include page = "uri of next page" />`

- used for request dispatching
- include scenario
- Contents are included, in run time (request processing)
- indicates request scope.

3) Nested tag - forward / include

`<jsp:param name = 'param-name' value = 'param-val' />`

- adds specified request parameter in the current request.

* JSTL (JSP Standard tag library)

- JSTL is a part of JavaEE Specifications
- its support exists in form of JARs | jakarta jars

Development Steps

- 1) Copy JSTL jars in your build path copy jstl jars in <tomcat-home>
- 2) Use taglib directive to import ~~any additional~~ JSTL supplied core tag lib.

```
<%@ taglib url="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

- 3) Invoke JSTL tag action.

- 3.1) tag for setting the attribute under specified scope

```
<c:set var="attribute name" value="static/dynamic val" %>
```

```
Scope="page|request|session|application" />
```

(default scope = page)

- 3.2) tag to remove an attribute from specified scope.

```
<c:remove var="attribute name" scope="page|request|session|application" />
```

- 3.3) a basic iteration tag

```
<c:forEach var="loopLocal Attribute" items="Iterable Src- List | Set | Map ..." %>
```

Body of foreach loop

```
</c:forEach>
```

- 3.4) if, for conditional rendering

```
<c:if test="boolean value">
```

body - evaluated in case ~~that~~ test = true

```
</c:if>
```

3.5) For SendRedirect + URL Rewriting

```
<c:redirect url="..." />
```

→ just like `<jsp:forward>` it also provides a child tag.

```
<c:param name="..." value="..." />
```

For sending additional parameters.

3.6) `<c:url>` for URL rewriting (to be used in client pull, in case user taking an action, clicks a links or button)

```
<c:url var="..." value="next.jsp" scope="..." />
```

3.7) Switch Case - n default case:-

Syntax:-

```
<c:choose>
  <c:when test="cond 1">
    :
  </c:when>
  <c:when test="cond 2">
    :
  </c:when>
  <c:otherwise>
    { default case statement }
  </c:otherwise>
</c:choose>
```


Hibernate

* What is Hibernate ?

- Complete Solution to manage automatic persistence in Database in java
- ORM (Object-Relational-Mapping) Tool.
- Most Popular JPA Implementor.

* JPA : Java Persistence API

- a part of Java EE / Jakarta EE Specifications (Interfaces)
- package:- javax.persistence / jakarta.persistence

* JPA gives Specifications → Hibernate is implementor of these specs.

(Other JPA Implementors, iBatis, Kodo, EclipseLink, Toplink, JDO)

* Spring Boot Framework :- default persistence provider = Hibernate

↓
(Founded by Gravins King, Open Source)

* Hibernate sits as a Intermediate Layer between Java app & DB.

* Why Hibernate ?

- Open Source & light weight
- Supports cache (L1, L2, query cache) for faster performance.
- auto table creation
- Simplifies JOIN queries
- Supports 100% DB independence.

(Hibernate will translate DB independent queries (HQL - hibernate query lang) / JPQL)

- Hibernate developer doesn't have to go to DB level, DB tables, cols, rows, sql.

(Hibernate is an abstraction layer above JDBC, i.e. hides the complexity of JDBC)

- JDBC → new separate connection per call to DriverManager.getConnection().
- Hibernate:- Creates a 'internal connection pool'
(DBCP → database connection pool → collection of ready to use db connections)
↓
When hibernate does this?
→ at hibernate booting time
- Exception translation mechanism
↳ Hibernate translates checked SQL Exceptions ---> unchecked hibernate exceptions, so programmer is not forced to handle the same.
- Hibernate Supports method chaining.

Hibernate Architecture

Hibernate Blocks (org.hibernate package)

JDBC | JTA

* Blocks

1) Org.hibernate.Session :- Interface

- Implementation classes are in hibernate core jar.
- it's abstraction (i.e. Super interface :- jakarta.persistence~~3~~EntityManager)

Session:-

- main runtime interface used for CRUD ops.
- Supplies CRUD API's (persist, get, merge, remove)
- Session represents a thin wrapper/abstraction around the pooled out DB connection & implicitly associated with L1 Cache.
- Light Weight Object, Thread un-safe Object, modifiable / mutable object.
- No need for accessing the session in Synchronized manner, since different thread representing different client requests, will have their ^{own} ~~an~~ Session Object.

2) Org.hibernate.SessionFactory (Interface)

JOB → to provide session objects (getCurrentSession)

- Singleton instance per DB / application.
- immutable, inherently thread safe.
- heavy-weight (time consuming)
- It has L2 cache associated with it (must be configured explicitly).

3) Configuration → org.hibernate.cfg.Configuration (Class)

- class act as a provider of SessionFactory.

4) Additional API's.

- Transaction, Query, CriteriaQuery, ----

* Hibernate Dev Steps

- 1) import hibernate template
 - download hibernate dependencies
- 2) Configure hibernate.cfg.xml
 - src/main/resources (build path)
 - DB configuration + more properties.

- 3) Create HibernateUtils class
 - build SessionFactory

code:-

```
public class HibernateUtils {  
    public static SessionFactory sf;  
    static → static {  
        sf = new Configuration() // empty config  
            .configure() // loads the config from hibernate.cfg.xml  
            .buildSessionFactory();  
    }  
} → (write a getter here)
```

- ensures Singleton instance of SF (SessionFactory)
- created at class loading time
- extends AutoClosable.

- 4) Create main class
 - to test bootstrapping of hibernate.

- 5) Create Hibernate managed POJO (entity class)

- Hibernate will manage entity life cycle.
- 2 ways for giving instructions

→ 5.1) XML - legacy way. (Deprecated)

5.2) annotations

↳ majority will be imported from JPA.

Imp. prop (interview ques.)

- * hibernate.dialect
 - used by hibernate to translate HQL or JPQL to DB specific SQL queries.
 - (deprecated in hibernate 5 onwards)

- * hibernate.hbm2ddl.auto → update.
 - (hibernate mapping) (data definition lang)

→ hibernate uses this property for ORM

→ With "update", it'll create new table if none exists for the entity class

→ otherwise continues with existing one

→ used in dev phase

* Recommendation

- ID property in Entity class, (which maps to primary key), should be in Reference Type (Long | Integer) (** hibernate is better at checking for null instead of 0 in primitive)
- Provide default, parameterless constructor
- Add getters & setters

* Mandatory annotations

① @Entity

- class level annotation, to declare entity class

② @Id

- can be added at field level | property (getter) level
- adds primary key constraint, at DB column.
table.

6) Add <mapping> entry in hibernate.cfg.xml

- 1 entry each Entity needs to be added.

```
<mapping class = "com.healthcare.entities.User" />
```

→ F.Q Name

7) Run TestHibernate

- as java application
- getSF → class loading (by JVM) → static block runs → new configuration created
→ configure() → Hibernate checks if table already exists, if none creates it.

* Recommended Annotations

3) @Table (name = "users") - to declare table name.

4) @GeneratedValue - automatic Id generations (autoincrement) (default id generation strategy is selected as AUTO)



- to specify auto-increments, we do (GenerationType = IDENTITY)
@GeneratedValue (Strategy = GenerationType.IDENTITY)

5) @Column (name = "user_id") - to specify column name



attribute = length → to specify size of column datatype.

@Column (name = "user_id", length = 25)

- 6) @Column (unique = true/false) → add unique constraint
- 7) @Column (nullable = false) → adds not null constraint.
- 8) @Enumerated (EnumType.STRING) → col type :- enum
- 9) @Lob → to specify long blob for col. type (can accept data till 4GB)
(mysql specific)
- 10) @Transient → skip from persistence, i.e. no column generation.

8) Create Hibernate Based DAO layer (for CRUD)

- DAO Interface
- DAO Implementation class.

* Generic Steps in Hibernate Based DAO.

1) Get Session from SessionFactory

- public Session getCurrentSession() throws HibernateException

↓
this checks if there is any session present on thread, if not
then it creates a new one session & returns it else
if already present then returns that session.

Session has 2 things → L1 Cache

→ Pooled out DB connection.

```
Session session = sessionFactory().getCurrentSession(); // new session
Session session2 = _____ // return existing
                               // session for same
                               // thread.
System.out.println(session == session2) TRUE
```

2) Begin a transaction

- Session API

public org.hibernate.Transaction beginTransaction() throws HibernateException.

- Transaction tx = session.beginTransaction();

```

3) try {
    // CRUD - API
    // Success → Commit
    tx.commit();
}
catch (RuntimeException e) {
    if (tx != null)
        tx.rollback();

    throw e;
}

```

* Ways of getting Session from SessionFactory.

① openSession ② getCurrentSession

- Irrespective of the fact that session exists or doesn't exist a NEW session obj. is returned, to the caller.
- Programmer MUST explicitly ~~close~~ close the session → `Session.close()` using finally block.

- If Session Object exists then, existing session is returned, otherwise new session is created & returned

- Session is auto-closed -- upon tx boundary.

Commit Rollback

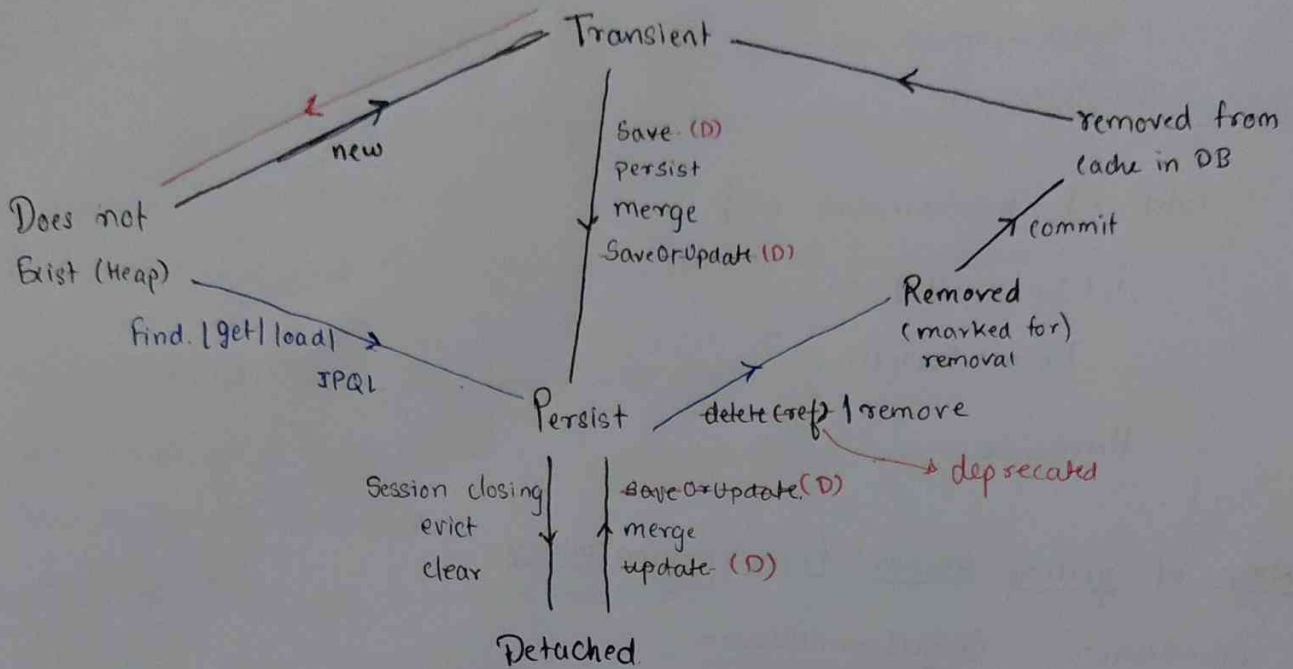
* Session API

① To insert a record

- `public void persist (Object TransientEntity)` throws `HibernateException`
- Transitions the entity, from **TRANSIENT** → **PERSISTENT**

* Entity State Transitions

L1 → L1 cache stores only reference of the object, not actual object



States	Does it exist in L1 Cache?	Does it exist in DB?
Transient	NO	NO (exists only in heap)
Persistent	YES	Rec will be inserted upon commit (transient → Persistent)
Detached	NO	Does Not Exist → Persistent (in heap) YES
Removed	YES (Session.remove) (Simply marked for removal)	YES (Simply marked for removal)
	NO (upon commit)	NO

* What happens internally on commit ?

→ 1) Hibernate invokes `session.flush()`

- here hibernate triggers, Automatic Dirty Checking.

i.e. it compares state of L1 cache (persistence context) with that of the database

INSERT → a) New Persistent Entity b) Updated state of persistent entity → UPDATE
c) Persistent Entity marked for removal → DELETE

2) `Session.close()`

- L1 cache is destroyed → (cache of references of entities) from java heap.

- pooled out DB connection returns to DBCP.

- Session Object is destroyed from heap.

* What happens internally on rollback ?

→ directly calls the `Session.close()` method; Same as above.

* To get Entity Details by its Id.

→ `public <T> T deprecated get/find/load (Class<T> class, Object id)` throws `HibernateException`.

T → type of entity (POJO)

returns → null in case of Invalid id.

OR PERSISTENT ENTITY ref.

Ex:- `User user = session.find ( User.class, world );`

`user = Session.load ( User.class , world );`

- ~~`user = session.get ( User.class , world );`~~ → `get` is deprecated

* `Session.find` Finds the user in L1 cache 1st, (If we provide valid ID)

if user exist:-

- then hibernate will return the user & not fire the repeating query.

if user not exists:-

- then hibernate fires select query & stores the result in the L1 cache.

→ If we provide invalid id, then db will fire all the queries provided as, the user is not available in L1 cache, each time.

(Imp)

Session.find (userclass, world)

checks in L1 cache, if Entity exists

NO

YES

- Immediately fires select query
- Processes the resultSet
- if not empty

- invokes default constructor of entity class
- directly sets fields using setters (ORM)
- adds its ref to L1 cache.
- returns PERSISTENT entity to the caller

- if ResultSet empty
- returns null.

loads the entity from L1 cache.

* Get All Users

o/p → display all user details.



Here enters JPQL.

Steps

1) Replace database specific SQL by database independent object oriented query language.

- HQL or JPQL

Here table name is replaced by POJO class name, col names by POJO property names. ? is supported but better recommendation is to use IN parameters.

e.g. Sql - Select * from User;

hql → From User

JPQL → Select u from User u;

u → POJO alias (reference)

u → u.* ⇒ all the properties encapsulated in User Object.

2) Create a Query Object to wrap your HQL / JPQL

- API

↳ public Query<T> createQuery (String jpql/hql, Class<T> result) throws
HibernateException.

T → type of the result.

3) API of org.hibernate.query.Query<T> → i/f

3.1) To execute the select query with multiple results.

public List<T> getResultList () throws HibernateException

T → type of result

Returns list of PERSISTENT Entities.

3.2) In case of single Result

public T getSingleResult () throws HibernateException.

Throws:-

- NoResultException → in case of no results
- IllegalStateException → in case of DML
- NonUniqueResultException → in case of multiple results

3.3) Same as getSingleResult, just returns a null value, instead of Exception

public T uniqueResult () throws HibernateException

Returns:-

the single result or null if there is no result to return

3.4) To get stream of results & further processing (filter, map, reduce)

```
public Stream<T> getResultStream ()
```

- Returns:-

functional Stream<T> of results

* Get all users details from specified Role, born after specified Date

i/p: role & date

o/p: list of selected users.

API of Query i/f → to set the named IN Parameters

```
- public Query setParameter (String paramName, Object paramValue)  
throws HibernateException.
```

(Used for setting IN Parameters)

Ex:- ~~Get all users~~ ~~from specified role~~

```
String jpql: "Select u from User u where u.userRole=:role and  
u.dob=:date";
```

```
List<User> users = Session.createQuery (jpql, User.class)
```

```
• setParameter ("role", userRole)
```

```
• setParameter ("date", userDob)
```

```
• getResultList();
```

* Display last names of all users from specified roles.

```
String jpql: "Select u.lastName From User u where u.userRole=:role;
```

```
List<String> lastNames = Session.createQuery (jpql, String.class)
```

```
• setParameter ("role", userRole)
```

```
• getResultList();
```

* Get fname, lname, dob of all users for specified Role.

* Here we need to create a customized view of the resulting Object, i.e. by creating a DTO.

```
List<UserDTO> list = null;
```

// JPQL Constructor expression

```
String jpql = "Select new com.sunbeam.dto.UserDTO (u.fname, u.lname, u.dob)  
from User u where u.userRole = :role";
```

// Session creation & begin transaction.

```
list = session.createQuery(jpql, UserDTO.class)
```

```
.setParameter("role", userRole)
```

```
.getResultList();
```

```
tx.commit();
```

* To Load Particular data from database (Some columns)

→ Use DTO Pattern

- used while presenting a view (projection - some cols lifted from DB)
- used while presenting aggregate data lifted from multiple tables

Steps

1) Create DTO class

- fname, lname, dob
- add default ctor + add matching parameterized ctor
- getters & setters
- toString() for debugging

2) Use JPQL constructor expression (if required)

e.g. "Select new com.sunbeam.dto.UserDTO (u.fname, u.lname, u.dob) from
User u where u.userRole = :role"

3) Create Query $\rightarrow$ Set IN Parameters $\leftarrow$ execute Query

$\downarrow$
List<User DTO> $\leftarrow$ get Result List

(Field Names in Entity Class & DTO class MUST MATCH)

* Display user details by specified ids.

i/p: list of ids (e.g. List<Long> idList)

o/p: list of users.

Imp

Detached vs Persistent

1) Entity is NOT present in L1 cache, has corresponding record in DB

2) The changes made to the detached entity are not synchronized (sync'd) with DB.

1) Entity - part of L1 cache
(transient → Persistent rec is inserted upon commit)

(no obj. heap → persistent, rec exists in DB)

2) The changes made to persistent entity are tracked by hibernate and applied @ commit.

* Evict & Clear

* public void evict (Object persistentEntity)

- Detaches (Persistent → detached) specified Persistent Entity from L1 cache

* public void clear ()

- detaches all persistent entities from L1 cache.

- DB connection DOES NOT return to DBCP.

- L1 cache is Not yet Destroyed. (empty state)

* Change user password having (email & pass)

ip: email & pass op: String

jpql = "Select u from User u where u.email = :email and u.password = :pass";

// Session & transaction, creation & begin

user = session.createQuery (jpql, User.class).setParameter ("email", email).

• setParameter ("pass", password).getSingleResult();

// User - PERSISTENT → set new Pass

user.setPassword (newPass);

tx.commit(); → now this will call session.flush()

↓
Dirty Checking → Update entity State

↓
Session.close() ← Fires DML Query

↓
L1 cache cleared & DB connection ret'd to pool.

* Apply discount in registration amount of user born before
"a date of specified role."

ip: discount and reg-amount and role

op: message (string)

→ In case of bulk updation, use update JPQL instead of
Select + n no. of update queries

1) Update JPQL

- Update User u Set u.regAmount = u.regAmount - :discount
where u.userRole = :role and u.dob < :date;

2) Use session API

- public Query <T> createMutationQuery (String update/delete JPQL)
throws HibernateException

3) Set IN parameters

- setParameters (String paramName, Object value)

4) Use

Query API :-

- public int executeUpdate () throws HibernateException

→ Returns no. of updated / deleted rows.

* Hard Delete (un-subscribe) user details

i/p: user id

o/p: message

JPQL:- ~~8~~ "delete User u where u.userId = :userid ;

int executeCount = session.createMutationQuery (jpql)

• setParameter ("userid", userId);

~~getSingleResult ();~~

• executeUpdate

* Upon commit() → delete query fired → removed User entity from DB & L1 cache

↓
Session.close()

Persistent Entity → After removing from DB, it's marked for delete (removed from L1 cache) → makes # of references = 0 → Removed (GC destroys)

* What will happen a non-transient Entity to persist method? (detached)

- Hibernate internally checks the Id of the entity, for NULL.
 - if the entity's id is NULL, that means the Entity is in **Transient** state.
 - if the entity's id is NOT NULL, i.e. the Entity is in **DETACHED** state.

And, if we pass a non-transient entity to persist method, it throws **PersistentObjectException**.

(i.e. we were using Id as a reference type, because it's good for null checking)

* If we want ~~from~~ go from **TRANSIENT** → **PERSISTENT** or **DETACHED** → **PERSISTENT**

→ We use the merge method,

- when entity is in **TRANSIENT** state and we use merge method, then hibernate fires the **INSERT** method/Query.
- when entity is in **DETACHED** state, then hibernate checks and finds Id property of Entity is NOT Null, then it fires the **Update Query**.

* If we directly fire a JPQL (UPDATE | DELETE) query,

- hibernate bypasses the L1 cache,
- (cannot cascade any operation to the associated entities)

E.g. → Delete a category

↓ cascade to products

JPA (Hibernate) should cascade a delete operation to Products Table (i.e. delete product records/rows of deleted category & then delete the category)

- Can't support Optimistic locking on the associated ~~related~~ entities.

(Better Alternative for Single Updation / Deletion)

→ Select followed by

↓

Update / Delete

* Hard Delete Product Details by name.

ip: name

op: Product

Product p = session.createQuery("Select p from Product p where p.name = :name, Product.class).setParameter("name", pname).getSingleResult;

session.remove(p); *// marked for removal (but present in L1 cache & DB)*

tx.commit(); → session.flush() → auto dirty checking → DML → Delete

↓
session.close();

New Topics

* Image Handling

1) Save User image in DB.

ip: imagePath, userId

Op:

→ as userId is provided, we can use the find method to get a persistent entity, instead of JPQL.

→ Now we have imagePath, but its datatype is byte[].

So, we need to get a byte[] from imagePath, using FileInputStream.

```
FileInputStream fin = new FileInputStream(imagePath);
```

```
byte[] image = null;
```

```
while (!fin.read()) {
```

```
    // append to image.
```

```
}
```

→ File Handling utility from apache supplied commons-io.jar

- add dependency in pom.xml (already added)

Methods

1) public static byte[] readFileToByteArray (java.io.File) throws IOException

- Read the contents of a file into a byte[] array. The file is closed afterwards.

2) public static void writeByteArrayToFile (File file, byte[] data) throws IOException

- Write a byte array to a file creating the file if it does not exist. close the file later

→ Code:-

```
File file = new File(imagePath);
```

```
if (file.isFile() && file.canRead())
```

```
    byte[] data = FileUtils.readFileToByteArray(file);
```

```
    // get user from its id
```

```
User user = session.find(User.class, userId);
```



```

    if (user != null) {
        user.setImage(data);
        msg = "image saved";
    }
}

```

```

tx.commit();
} → try block ends

```

```

catch (RuntimeException e) {
    if (tx != null)
        tx.rollback();
    throw e;
}

```

2) Restore the image from DB

ip: user email, new image file path → where image from DB will be stored.

op: BLOB/string

* String jpql = "Select u from User u where u.email = :email";

try {

```

    User u = session.createQuery(jpql, User.class)
        .setParameter("email", email)
        .getSingleResult();

```

```

if (u.getImage() != null) {

```

```

    FileUtils.writeByteArrayToFile(new File(imagePath), u.getImage());

```

```

    msg = "image saved restored";

```

```

}

```

```

tx.commit();

```

```

} → try ends

```

```

catch ( ... ) {

```

```

    ...
    ...

```

```

}

```

* Lombok (Annotations Library)

- Annotations library, to reduce boilerplate code from java classes / entities (mainly from POJO layer)

Steps

- 1) Add lombok dependency, in pom.xml.
- 2) Install lombok library to IDE's location.
- 3) Use class level annotations - Compile time

Ex:- @NoArgs Constructor

@AllArgs Constructor

@Required Args Constructor → will create a parametrized ctor including only final fields.

@Getter, @Setter

@ToString (exclude = {....}, callSuper = true)

@Data - typically do not use for entity classes (can be used with DTO's)

@Equals And hashCode (of = {.....})

↓

- on which field you want to override methods & by default considers all fields.

@Slf4j - Simple logging facade for java & m

& many more.

Advanced Hibernate (Project Specific Technology)

→ In Healthcare ER dig.

- We see Id field, createdOn & lastupdated fields repeating.

→ So, Instead of Repeating common fields in multiple classes, apply Inheritance.

1) Create Base Class (either abstract or concrete)

- all other entities can extend from it
- No separate table required for it
- Add Common fields.

Example :- Id, creation date, updation TS, version field (used in optimistic locking)

Annotations Used

@MappedSuperclass

- JPA class level annotation
- It represents a common Base class, which contains common fields & all other classes are extending from it & no need to create a table for it.

Hibernate annotations (Field | property - getter)

@CreationTimestamp

- LocalDate | LocalTime | LocalDateTime

@UpdateTimeStamp

- LocalDate | LocalTime | LocalDateTime

Use case of @Mapped Superclass

@Getter @Setter @ToString

@MappedSuperclass

```
- public abstract class BaseEntity {
```

@Id

@GeneratedValue (strategy = GenerationType.IDENTITY)

private Long id;

@CreationTimestamp @Column (name = "created-on")

private LocalDate createdOn;

@UpdateTimestamp

private LocalDateTime;

Similarly here change col name.

}

★ ★ ★ ★ ★
★ Now in User class ★ (Imp)

★ ★ ★ ★ ★

As we have to set the column name of id as userId, we use another attribute Override annotation.

@AttributeOverride → Imp

1) usage example.

@AttributeOverride (name = "id", column = @Column (name = "user-id"))

↓
inherited attribute name

↓
name which needs to be set as final col name.

* Association between Entities

- Instead of choosing Patient IS-A User (i.e. Inheritance)

↓ we do

- Patient Has-A User (i.e. Association)

Why?

- In order to extend this to microservices (MS) based Architecture, where patient MS (microservice) can be designed independently from UserMS.

- Meaning there will be different from DB for different MS.

- In this case, Since association (aggregation - weaker association) will be easier to handle rather than inheritance.

Eg. Doctor Has-A User

(Doctor 1 $\rightarrow$ 1 User)

* Uni Directional Association Between Entity's

$\rightarrow$ uni-direction one-to-one association between Doctor and User.

Doctor $\rightarrow$ User

$\rightarrow$ uni-direction one-to-one association between Patient and User.

Patient $\rightarrow$ User.

(Directionality only exists in java)

* Design Choice :-

- depends upon project's data navigation requirements, i.e. if you need to access user details from Patient or vice versa.
- In our healthcare domain,
Patient entity will be accessed ~~by~~ more often to access user details,
(Patient 1 $\rightarrow$ 1 User)

NOTE :-

There is no rule (explicit) in one-one association.

We can configure any ~~association~~ side as the owner of the association.

$\therefore$ What's the Owning Side?

In unidirectional one-one association, the side containing the physical mapping (foreign key) is considered the Owning Side of the Association.

Eg. Patient Has-A User

- In Patient Class - extends BaseEntity
- familyHistory.
- bloodGroup.

+

private User userDetails;

→ Configure Patient as Owning Side :-

(Project Tip - Exclude ~~always~~ association fields from toString())

↳ it'll cause recursion in bidirectional association

* Problem (Imp)

- After adding associations, hibernate throws MappingException,

Cause

- Hibernate can't figure out type of (E-R) (i.e. is it one-one or many-many)

Solution

- add annotations

Mapping

i) @OneToOne

ii) @OneToMany

iii) @ManyToOne

iv) @ManyToMany

* Problem :-

- Name of foreign key was decided by Hibernate.

→ To Customize it

- @JoinColumn

- attribute :- (name, nullable, etc...)

Ex:- @JoinColumn (name = user_id, nullable = false)

private User userDetails;

* Property in hibernate.cfg.xml

(hbm2ddl.auto = create) → hibernate drops ~~and~~ existing tables and creates new tables upon application restart, while sessionFactory.

→ used in development Phase only ?

* Appointment Entity

- extends BaseEntity
- add Specific fields

* Many-to-Many, Uni-directional Association

(Between Patient $\rightarrow$ DiagTest)

Project Tip $\rightarrow$ Use Collection Type - as Set, in case of Many-Many associations

$\rightarrow$ Use Collection Type - as List, in case of One-Many associations

~~$\rightarrow$ Always initialise the~~

$\rightarrow$ While declaring a collection - Always initialise the collections (to Avoid **NULLPOINTEREXCEPTION** while adding/removing the element).

Problem:- Hibernate Throws MappingException

Cause:- It is unable to figure out entity or non-entity & doesn't know many-to-many or one-many.

Solution:- Adding mapping annotation

@ManyToMany

- private Set<DiagTest> diagTests = new HashSet<>(),

$\downarrow$

- must need to override hashCode & equals method

- using lombok annotation.

@EqualAndHashCode (callSuper = false, of = "name")

* To customize name of join table & column names

@JoinTable (name = "patient_test", joinColumns = @JoinColumn (name = "patient_id"),
inverseJoinColumns = @JoinColumn (name = "test_id"))

Problem - org.hibernate.PropertyValueException

Cause:- No link established b/w Patient & User

Solⁿ:- Establish the link b/w entities (Object world)

- call setter for setting userDetails
- but ~~do~~ even before persisting Patient we have to persist the user / userDetails in 1st cache.
- So, first do , `session.persist(patient.getUserDetails);`
`session.persist(patient);`

But hibernate provides an annotation property at JPA level.

`@OneToOne (cascade = CascadeType.ALL)`

* Even after setting the Link

Problem:- org.hibernate.TransientPropertyValueException :- Not Null Property references a transient value

* Cascading

* Instead of Explicitly saving parent record first and child record later, Hibernate can perform this automatically. ?

→ Using Cascading (in the JPA Level not the table level)

* What is it ?

→ It is the ability to propagate any changes from Source Entity to Associated Target Entity. (Patient → User)

→ To simplify DAO layer & to reduce boiler plate code.

(Not to add in Many-One Relation.)

* How ?

→ use attribute of @OneToOne, @OneToMany, @ManyToOne

Cascade = CascadeType[] : data type

CascadeType : Enum in JPA

With constants: ALL, PERSIST, MERGE, REMOVE, REFRESH, DETACH

Ex:- In Patient class

@OneToOne (cascade = CascadeType.ALL)

@JoinColumn ...

private User userDetails;

* To add a unique constraint over multiple columns

@Table (name = "appointments", uniqueConstraints = @UniqueConstraint
(columnNames = { "doctor_id", "app_date_time" }))

* Selecting an appointment if Scheduled

→ SELECT a.id From Appointment a where a.myDoctor.id = :docId
and a.sts = :status and a.appDate Time BETWEEN :start and
:end;

* Hibernate does not Support Nested Transaction. (Chutiya)

Problem:- Exception From BookAppointment to checking Doc's Availability
Hibernate Throws exception - TransactionException

Cause:- Hibernate Does Not Support Nested Transactions.

Solution:- Begin only one transaction in BookAppointment & Continued with the
Same transaction in checking doc's availability.

* View upcoming appointments For Patient

ip: patientId

op: List of Scheduled Appointments

SQL:- Select a.appointment_id, a.app-date-time, u.first-name, u.last-name,

JSQL:- Select a.id, a.app-date-time, u.firstName, u.lastName, from Appointment a
↓
Field name
Join a.myDoctor d and a.app-date-time > :ts and a.myPatient.id = :pid
order by a.app-date-time asc;

* Bi Directional Association

Doctor 1 $\longleftrightarrow$ * Appointment

In Doctor Class,

- private List <Appointment> appointments = new ArrayList();

Problem:- In above bi-dir association (Doctor 1 $\longleftrightarrow$ * Appointment)
Hibernate creates a Extra JOIN TABLE, with doctor-id & appointment-id
as composite * PK + FK.

Cause:- In a bidirectional association, hibernate can't figure out

- which is the owning side (having FK)
- which is the non-owning side (inverse side) (not having FK)

In One-many association

→ One - inverse (non-owning)

Many - Owning

- Add mappedBy in @One To Many

E.g.

@OneToMany (mappedBy = "name of the field as it appears in owning side :
myDoctor", cascade = CascadeType.ALL) :

Problem:- Hibernate throws, LazyInitializationException, while accessing
appointment details

(∵ we are trying to display doctor + list of appointments in bi-dir
association)

Cause:- JPA (Hibernate) supports default data fetching policies

one - one = EAGER

one - many = LAZY

many - one = EAGER

many - many = LAZY

In our case, we have One $\rightarrow$ Many (LAZY)

$\rightarrow$ means, when you fire select query on Doctors table using (Session.find)
Hibernate does not auto fetch data from appointments table.

Why?

$\rightarrow$ for better Performance

- Hibernate uses a Proxy Pattern (using byte-buddy jar), to represent
• un fetched data from DB

- Anytime, you are accessing un-fetched data (proxy), outside the session
scope

- Hibernate throws LazyInitException

Solution:-

1) ~~Change~~ Entity Layer Solution

$\downarrow$

- Change Fetching Policy (Lazy $\rightarrow$ Eager)

- Add Fetch = FetchType.EAGER in @OneToMany

Use case:- when the size of many is small.

disadv:- if size of many is big, it gets out of our control

2) Keep Fetching Policy Lazy (Default) $\rightarrow$ DAO Layer Solution

Here, we separately fetch details of associated entities.

$\downarrow$

i.e. Fetch doctor + fetch appointments.

- Simply access the size of collection, within session scope
(ie. before commit)

disadv:- Getting complete data (doctor + appointments) in multiple queries

3) Use "Join Fetch" in JPQL.

JPQL - Select d from Doctor d $\downarrow$ join fetch d-appointments where d.id = :docid.
left

→ We do left join, ^{so} if the doctor doesn't have a single appointment, then the Hibernate doesn't throw an Exception.

JPA VALUE TYPES (Imp)

Value Type

- Does not have an independent existence (i.e. life cycle)
- Embedded within the owning entity
- doesn't have its PK
- The association b/w Entity → Value.

Basic Value Type

- built in java types
- e.g. Person Has-A firstName
- String, Date, Primitive types
- Optional - @Basic

Composite Value Type

- User Defined Type
- e.g. Person Has-A ~~Address~~ Adhaar card
- Mandatory Annotation
- @Embeddable - class lvl annotation over the value type
- can't add
- @Entity @Id @Table
- while adding it in the entity
- @Embedded (optional annotation)

Collection Type

- collection of basic value types
- e.g. Emp Has-A Set<Skill> @SkillSet.
- collection of embeddables
- (Person Has-A list of paymentOptions)
- Mandatory Annotation
- @ElementCollection
- @CollectionTable
- Optional annotation to declare collection of Value types.

* MVC (Model View Controller)

- Standard industry grade design pattern, meant for clear-cut Separation of concerns (tasks or responsibilities)

- In custom implementation of the MVC pattern:-

1) Model

- Business Logic Layer + data (Service Layer)
(POJO)

2) View Layer

- JSP / Thymeleaf / Velocity / React / Angular / Vue.js / NextJS

3) Controller

- Typically a Servlet (used in Spring MVC) or a filter (used in Struts 2 Framework)
- manages
 - navigation
 - optionally manages java beans (Service Layer)
- If Service Layer is absent
 - manages DAO layer.

(Along with MVC, we use another pattern called FC ↓)

* Front Controller

- additional design pattern, used along with MVC.

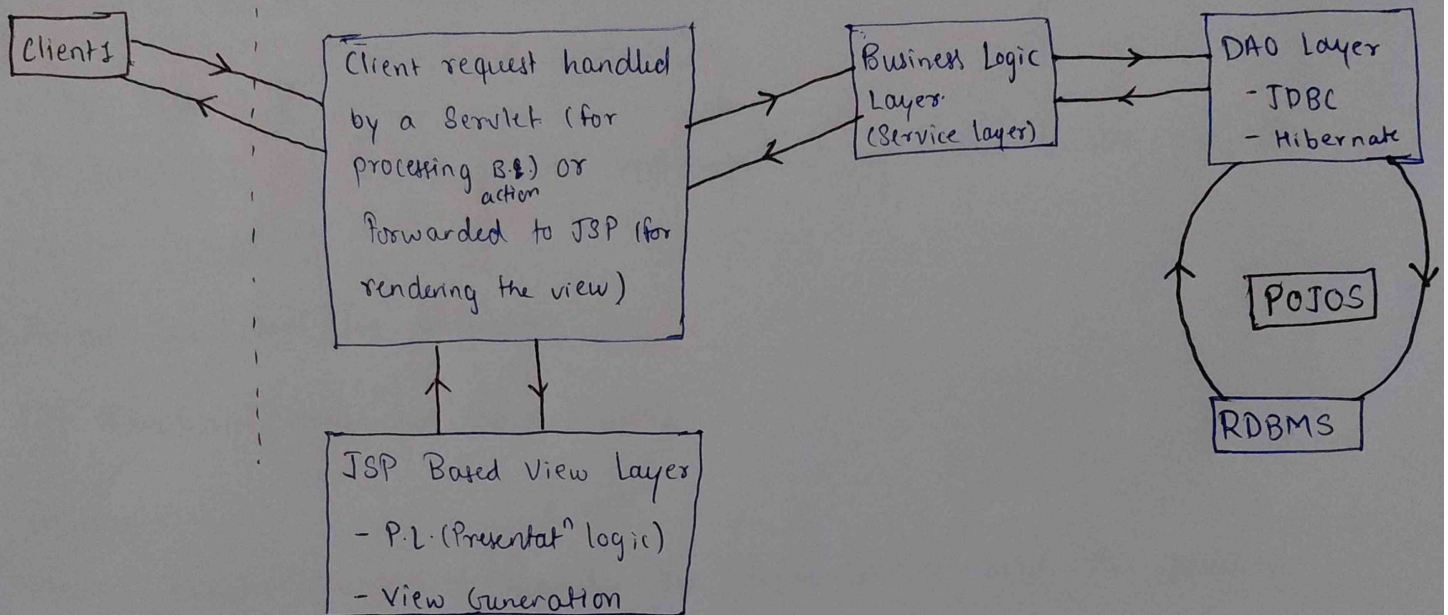
Job:-

- To ensure that any request coming from any client, is intercepted by a common component - Front Controller

works as → common gatekeeper / global interceptor (as a centralized dispatcher or entry point of MVC).

→ It will dispatch ~~request~~ client requests to further components, based upon nature of the request.

MVC Implementation



* MVC Development Steps

- 1) Create new project / continue with Healthcare - patient view
- 2) Good Practice
 - DO NOT GIVE DIRECT ACCESS TO VIEW LAYER TO USER/CLIENT.
 - copy JSPs under <WEB-INF>/views → add separate folder.
- 3) Configure welcome page as
 - <WEB-INF>/views/login.jsp
- 4) After Submitting the form

* HttpServletRequest Method (Exam Objective)

public HttpSession getSession (boolean ^{create} ~~session~~)

- if create == True, it works exactly like request.getSession, i.e. it returns the existing session
- if create == False & no session exists, method returns null.

Hibernate

(Imp. Project Tech.)

→ Maven

What is Maven? (build tool to manage dependencies)

- it's a popular automation and dependency management tool used mainly in java projects. (It helps in building, cleaning, compiling & later installing any java based projects.)

Why Maven?

- it simplifies the building and management of java-based projects.

Vendor- Apache

- Another build tool for project → Gradle

(In JavaScript/Node we have, npm)

In java:-

- Maven creates pom.xml to specify the dependencies and the plugins
↓
(just like package.json)

- Maven defines goal for project management,
 - Clean
 - compile
 - test
 - install
 - package.

- In Eclipse IDE (STS) → has an built in Maven plug-in (mee)

- dependencies → Maven dependencies

* Maven Project > - create a simple proj. (Skip)

↓
Artifact ID → project name.

Group ID → Package name.

packaging → jar

Name :- same as Artifact ID.

↓
Finish

* Maven Polder Structure

Automatically, `src/main/java` → main java files, Servlets, DAOs, etc.
Kept in runtime `src/main/resources` → configuration files.
Class Path `src/test/java` → testing classes written here
`src/test/resources` →

needs to be updated. ← JRE SYSTEM Library [1.8]

`src`

`target` → all the compiled ~~files~~ classes as well as JAR/WAR files.

`pom.xml`

java
application
archive

web
application
archive

* hibernate groupId & artifactID

groupId :- org.hibernate

artifactID :- hibernate-core

Spring

- it simplifies java development.

* Why Spring ?

→ Simplifies overall java development

- Allows java developers to create loosely coupled applications, Spring manages the dependencies, so that dependencies objects are completely de-coupled from the dependencies.

- Supplies ready made implementation of design patterns (Singleton, Factory, proxy, MVC, front Controller & many more)

Spring support the MVC pattern for developing monolithic web application as well as ^{RESTful} web services.

- Reduces Boilerplate Code.
- Helps to build Easy, Simple, lightweight applications.
- Excellent support for JDBC as well as ORM, along with automatic transaction management support.
- Provides smooth integration with other frameworks such as Hibernate, Struts, EJB.
- Modular and non-intrusive design
- Supports AOP (Aspect Oriented Programming) for the separation & modularization of cross cutting concerns (= repetitive tasks) from the core tasks
- Supports Unit Testing as well as Integration Testing.
- Supports (Excellent) for building Microservices based architecture.
- Developing & securing web apps as well as Restful web services becomes easy.

* What is it ?

- It is a container, since it manages life cycle of Spring Beans.

What is Spring Bean ?

- It is a java class (Object) whose life cycle is managed by Spring Container (SC).

e.g. of Spring Beans → REST Controller, Controller, Service, DAO.

- It is a framework of frameworks.

- It is a modular & extensive framework, containing a lot of frameworks.

- Why it is a framework ?

- It provides ready made implementation of standard design patterns (MVC)

~~* What is Dependency Injection ?~~

- ~~- Background~~

- ~~- In the earlier programming~~

* Tight Coupling VS Loose Coupling

- In conventional/Traditional Programming model

- dependent Objects create/manage dependencies

- e.g. Servlet depends upon DAO Layer for Data Access Logic

in
So, ↑ the init() method - DAO instance was created.

If there are any changes in DAO Layer (i.e. Dependency changes)

You'll have to make changes in Servlet.

- If there are multiple dependent Objects using the same dependency, the app becomes brittle, difficult to manage.

(Example of Tight Coupling)

- Dependency Injection

- Design pattern to achieve de-coupling of dependents from dependencies.
- Instead of dependent objects managing dependencies.
 - A 3rd party manager/container (Spring Container SC) will provide the dependencies.
- Since the control of managing the dependencies, is not with programmers.
- It's called as Inversion of Control.

* We can still achieve loose coupling without Spring, but then we have to

- create dependent Obj.
- Create dependency &
- Create the wiring b/w them

All by ourselves, so for delegating these tasks we use Spring

*

Java Classes (Spring Beans)

Prog. supplies meta
data info --xml
or annotations
or config

meta
data

The Spring Container

How to Create Beans

How to Wire them

How to Manage Life Cycle.

Final
Result

Ready To Use
Application

PTO

* Exam Perspective Specific (No use in Project)

* Bean config xml file.

```
<bean id=" " class="F.Q. name of class" />
```

imp attributes of bean tag

name	Significance
id	- mandatory, beans unique identifier
class	- mandatory, F.Q. class name
Scope	- Optional, default value $\rightarrow$ Singleton - In a standalone main based app, only 2 scopes supported $\rightarrow$ Singleton Prototype - In a web based app, we have, - Singleton Prototype Request Session or Global Session.

* Singleton - Spring Container will create & share single bean instance for multiple reqs / demands.

* Prototype:- SC creates a new bean instance per request / demand.

* ~~lazy-init~~

lazy-init

- boolean attribute, default = false
- Applicable only to Singleton beans

init-method

- to specify the name of custom^{init} method
- default pattern of init method $\rightarrow$ public void anyName ()
throws Exception {init logic}
- It'll be automatically called by SC after D.I.

destroy-method

- to specify name of custom destroy method.
- It'll be automatically called by SC before d.i. of Singleton beans

- Nested Tags inside Bean Tag

- to support constructor based DI.

- `<constructor-arg name="myTeacher" value|ref="..." />`

↓
name of ctor
variable of bean

- value = for providing primitive type values

- ref = for providing reference type refs.

Singleton

1) Singleton instance of bean is shared across multiple requests made to SC.

2) Default loading policy = eager.
(can be changed to lazy using lazy-init)

3) SC will automatically invoke destroy static method upon context closing

4) Created as stateless beans (can't obtain the conversational state of the client)

Prototype

1) SC will create a separate bean instance per demand.

2) Prototype Beans are always loaded lazily (upon demand), lazy-init is not applicable.

3) SC doesn't invoke destroy static method.

4) Stateful beans can hold conversational state of the client.

Spring Container API

org.springframework.beans

FactoryBeanFactory (Super i/F)

- for creating SC

- BeanFactory - Provider of Spring Beans

O.S. Context. ApplicationContext (Sub i/F of BeanFactory)

- used in case + spring & web + spring

Implemented by O.S. Context-Support. ClasspathXmlApplicationContext

- Meaning - SC started from meta data instructions
- xml based, placed in runtime classpath

Implemented by O.S. C.A. AnnotationConfigApplicationContext

- Represents SC loaded using java config class + annotations

- ctor

↓

AnnotationConfigApplicationContext (

Class <?> config/component class)

Spring API

* How to get ready to use Spring Bean from SC?

API for BeanFactory

→ `public <T> T getBean(String beanId, Class <T> beanClass) throws BeansException`

T → Type of the Spring Bean

* Constructor Based DI is used to inject Mandatory Dependencies.

* Types of Wiring (Dependency Injection)

Explicit Wiring

- must supply ctors/setters/factory methods + XML configuratⁿ for beans

1) Constructor Based DI

- Provide Parameterized ctor
- Provide
`<ctor-arg name="..." value/ref />`
- Used for injecting mandatory dependencies (i.e. dependency is created first & dependent next)

2) Setter Based DI

- used for optional dependencies (i.e. dependent is created first & then dependency)
- Provide setters in dependent bean
- Provide
`<property name="..." value/ref />`
- one per dependency.

Auto Wiring

1) Constructor Based Auto-Wiring

- For injecting mandatory dependencies
- needs to supply parameterized ctor with arg/dependency in dependent bean class
- No need of `<constructor-arg>` tag
- SC tries to match data type constructor with data type of dependency bean.

What happens if?

a) no match / absence of match

- SC throws Exception

NoBeanDefinitionFoundException & aborts

b) in case of multiple matches

- SC throws

NoUniqueBeanDefinitionException & aborts

c) In case of exact match

D.I Succeeds:

3) Factory Based DI (least used)

- SC can provide dependencies ~~also~~ even w/o any parameterized ctor or setters, using factory method.
- Factory method
 - public static method returning bean Instance.
- Add Factory method in Bean class
- In XML,
 - add factory-method = "nameoffactorymtd" attribute in bean tag.
 - To supply args to the factory method

```
<constructor-arg name/ ref/value />
                  type /
                  index
```

* <bean autowire = "no | byName | byType | constructor" />

2) Setter Based Autowiring

- for injecting optional dependencies.
- needs to supply setters / dependency in dependent class.

- No need of <property> tags.

- In the absence of a match, SC still continues, But will cause **B-L Failure** (NPE xcp) (Null pointer excep.)

a) autowire = "byName"

- SC tries to match data type of the property (derived via setter) with dependency bean id.

b) autowire = "byType"

- SC tries to match data type of the property (via setter) with data type of dependency bean.
- In case of multiple matches - SC throws the Exceptⁿ **NoUniqueBeanDefinitionException** & SC aborts

Spring Bean LC (managed by ~~SC~~ S)

Start SC → auto done in web app by DispatcherServlet
i.e. Front Controller.

SC refers to XML based instructions / Annotations

SC checks the scope

Default Scope = Singleton

checks Lazy init

Default value = false
(i.e. Singleton & eager)

lazy-init = true
(Singleton & lazy)
Same LC invoked by
SC after 1st demand.

Starts the LC

(init sequence -- A)

Locate - Load - Instantiate

Can be either def. ctor / param. ctor for constructor based DI)

OR

(Setter Based DI (property tags) or SC invokes matching setters)

init-method

Supplied

- SC invokes init style method
(public void myInit() throws Exc... {})

Not Supplied

- no init method invoked

Spring bean is ready for servicing the Client

- B.L. Execution

@SC Shutdown → ctx.close()

SC checks destroy method.

Supplied

- SC invokes the destroy method & marks
bean for G.C.
- End of LC.

Not Supplied

- doesn't invoke destroy method callback
- marks bean for G.C.

(public void myDestroy() throws Exc... {})

* Hybrid Configuration Approach

Hybrid approach (Reduced xml + reduced java code + majority annotations)

Steps :- ① Remove all Bean Tags

② Enable class internal ~~const~~ annotations

<context: annotation-config />

- Specify Base package to scan for beans / component

<context: component-scan base-package = "com.cdac" />

———— x ——— x ———

XML Configuratiⁿ

1) <bean id class>

Annotations

1) Class Level Annotations

- @Component (generic spring bean, super type)
- @Controller - In MVC scenario, req. handling bean
- @Service - Service Layer Bean
- @Repository - DAO Layer bean
- @RestController - RESTful web service provider bean.
- @ControllerAdvice - global exceptⁿ handler for all controllers
- @RestControllerAdvice - global exceptⁿ handler for all ~~rest~~ rest controllers

2) scope attribute

- @Scope (value = "Singleton | prototype | request | session")
↑
(class level annotatⁿ)

3) lazy-init attribute

↓
- @Lazy (true | false)

4) init-method attribute

- @PostConstruct (method lvl annotatⁿ)

5) destroy-method attribute.

@PreDestroy

(method lvl annotation)

6) autowire = "by^{Type}~~name~~"

@Autowired

(setter method annotation)

7) autowire = "constructor"

@Autowired

(parameterized ctor annotation)

8) autowire = "byName"

@Autowired & @Qualifier ("dependency bean id")

9) @Autowired can be directly added over the field, Field-level D.I.

10) bean config xml file. ^{replaced by} ----->

@Configuration annotated java class

11) <bean> tag in xml file

@Bean annotated methods (to return a bean to managed by SC)

12) <name, value> to inject a value

@Value ("SpEL expr") - Spring expression lang.

13) <context:component-scan />

@ComponentScan

* When doing Field level DI, only writing the @Autowired won't inject the dependency, as SC will try to find Bean by the data type, instead of its bean id.

But to inject the dependency we need to give another annotation @Qualifier & now SC will search for the bean with ^{provided} name/id & inject it. ("dep.name")

* Spring Web Module

* SPRING MVC

1) either copy temp. proj. or create maven proj. from scratch.

- Force update maven proj.

- Build (Clean & Compile)

2) Configure Spring supplied front controller to intercept any request coming from any client. (Global Interceptor)

Class → org.springframework.web.servlet.DispatcherServlet

Add xml tags → in web.xml

Servlet-name : Spring

Servlet-class : org.springframework.web.servlet.DispatcherServlet

load-on-startup : 1

url-pattern : /*

2.1) Job of D.S. → to start Spring Container in a web application.

- using hybrid approach

- Defaults name

main-config file - ServletName-servlet.xml

Location - WEB-INF

Contents - SC instructions.

3) refer to WEB-INF \ Spring-servlet.xml.

<mvc:annotation-driven />

- Enable MVC annotation based programming

SC creates an empty

- for storing the mappings b/w incoming req & Handler (req. handling controller)

4) Objective

web client → http://host:port/context-path/

response → welcome message

5) Create Hello World Controller Class

- Configure it as Spring Bean

- Add Request handling method

- method level annotation

@Controller - class level annotation.

@RequestMapping - can incl.

Spring MVC Flow

② (SPRING BEAN) Singleton & Eager

Handler, Mapping - i/f
 - map of request mappings
 - Key = Value of the @RequestMapping
 @GetMapping | @PostMapping
 - Value = F.Q. handler class name, method name

③

(SPRING BEAN) Singleton & eager
 Request Handling Controller
 @Controller - class lvl annotatⁿ
 @RequestMapping | @GetMapping |
 @PostMapping - method lvl annotatⁿ
 Return - String | ModelAndView

Dependent - Handler
 Depcy - Service Layer
 @Autowired
 Service Layer i/f

key = "/"

Value = Handler + method

method invocation

String (LVN) or
 Model & View &
 String (redirect location)

①

U.S.W.S. DispatcherServlet
 - Front Controller
 - Spring Supplied

④

ViewResolver - SPRING BEAN
 (Singleton & eager)
 Props
 prefix - /WEB-INF/views
 Suffix - .jsp
 Job - translates logical view
 name to actual view name

Translate → AVN = pre + LVN + Suffix

Thin Client
 (web browser)

SC checks for
 results

Present
 Absent

Stores model attributes
 under request scope

SC forwards the client
 to AVN.jsp

View layer - jsp based
 (eg. index.jsp)

Depat → Service Layer
 Depcy → DAO Layer

Service Layer (B.I)
 @Service - Spring Bean
 @Transactional - auto
 Tx mgmt

DAO Layer (Data Access Logic)
 @Repository
 - CRUD Methods (w/o boilerplate
 code)
 - Thin Layer

JPA | Hibernate managed
 Entity, Value Types

RDBMS

Depdt → Hibernate based DAO Layer

Depcy → Sessionfactory (i/f)

@Autowired

Private SessionFactory i/f

@GetMapping - intercept Get requests only.

@PostMapping - intercepts Post requests only.

* @Controller

→ used to declare a spring bean containing request handling logic.

(Presentation Layer) Handler - Singleton & Eager.

* @RequestMapping("/")

→ used to map incoming request (GET|POST|...) with handler method which needs to be called on a request.

→ Optional but Recommended, to specify base url.

for example → "/test/test1"

@RequestMapping("/test")

↓
Class level
Annotation

↳ now we don't need to repeat for each controller method.

* How to share dynamic results from handler → View Layer ?

→ Using model attributes

* → ways to create model attribute/s

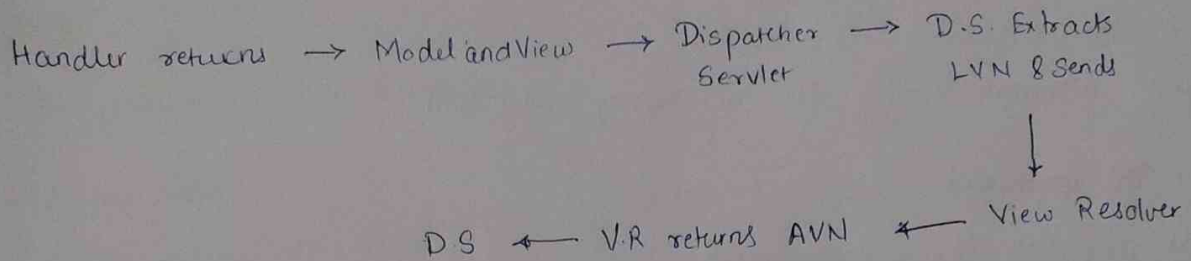
a) Using O.S.W.S ModelAndView class

- represents a holder that contains both results + Logical view name (LVN)

ctor:-

```
public ModelAndView (String viewName, String modelAttributeName,  
Object modelAttributeValue)
```

* Flow



- D.S. stores model attributes under request scope (request.setAttribute("nm", val))
- D.S. forwards the client to view (JSP)
- { requestScope.modelAndViewName }

❗ Disadvantages of this approach.

- no inversion of control, as we have to create the dependency.
- model and view are mixed together, they need to be separated.

② Use O.S.ui.Model - interface

- Represents map of model attributes
- can be configured as dependency.
- by adding it as method argument.
- Complete IOC approach.

Method of model, to add the attribute.

- public Model addAttribute(String modelAttributeName, Object modelValue)

* Spring, Hibernate Integration Steps (Without Spring Boot)

MVC

- import hibernate config file in Spring-Servlet.xml
- Edit DB password
 - db.properties (from <resources>)
- Details of Hibernate config file - to declare spring beans.
 - Configure Apache supplied DBCP (implements javax.sql.DataSource) for pooled DB connectivity.
 - DB config, pool min & max size.
 - Configure Spring supplied SessionFactory provider
 - O.s.Orm.hibernate5.LocalSessionFactoryBean
 - add hibernate props
 - packagesToScan : name of the base package of entities/beans
 - + showSql, dialect, hbm2ddl.auto
 - inject DBCP (DataSource) reference.
 - Configure Spring supplied Tx manager bean
 - class : O.s.Orm.hibernate5.HibernateTransactionManager.
 - To reduce repetitive (boiler plate) tx management logic from DAO layer.
 - inject LocalSessionFactory reference.
 - Enable annotation based Tx mgmt support.

Example

* Entity

Department 1 $\rightarrow$ M Employee

- DepartmentDAO
 - i/f & implementation class
 - List <Department> getAllDepartments();
 - impl. the methods
- DepartmentService
 - i/f & impl class
- DepartmentController
 - add a req. handling method.
 - to render a view containing list of depts
- HomeController

* Why we choose Spring Boot ? (Imp Interview Que)

1) Config efforts

- pom.xml (plugins + dependencies)
- web.xml (configure DispatcherServlet)
- spring-servlet.xml (instructions for Spring-container)
- Database Properties
- hibernate-persistence.xml
 - DS (DBCP), localSessionFactory, Tx manager bean

$\rightarrow$ To reduce Configuration efforts to Zero.

* Spring Boot

- Spring Framework (D.I / IOC)
- Embedded Tomcat Container
- w/o efforts in creating pom.xml & ^{creating} config files
adding explicit

* Problem → WhiteLabel Error Page

- while accessing department list (DAO Layer)
- error page generated by Spring Boot.

Cause:- Exception Occurs

- No CurrentSessionContext configured
- org.springframework.orm.jpa.JpaSystemException

Reason - Spring Boot doesn't give you direct $\downarrow$ SessionFactory & Session & access to native hibernate API's.

Solution:- Replace Hibernate by its abstraction - JPA (Specification)

- changes in DAO layer
- JPA based dao layer

→ class lvl annotation

@Repository

Dependency

@Autowired OR JPA annotation (@PersistenceContext)

private EntityManager manager; // Super i/f session

- In CRUD methods, directly use EntityManager's API